

Red Hat Data Grid 8.6

Data Grid Operator Guide

Create Data Grid clusters on OpenShift

Last Updated: 2026-06-03

Red Hat Data Grid 8.6 Data Grid Operator Guide

Create Data Grid clusters on OpenShift

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Data Grid Operator provides operational intelligence and reduces management complexity for deploying Data Grid on OpenShift.

Table of Contents

RED HAT DATA GRID 6

DATA GRID DOCUMENTATION 7

DATA GRID DOWNLOADS 8

MAKING OPEN SOURCE MORE INCLUSIVE 9

CHAPTER 1. DATA GRID OPERATOR 10
1.1. DATA GRID OPERATOR DEPLOYMENTS 10
1.2. CLUSTER MANAGEMENT 10
1.3. RESOURCE RECONCILIATION 11

CHAPTER 2. INSTALLING THE NATIVE DATA GRID CLI AS A CLIENT PLUGIN 12
2.1. INSTALLING THE NATIVE DATA GRID CLI PLUGIN 12
2.2. KUBECTL-INFINISPAN COMMAND REFERENCE 12

CHAPTER 3. INSTALLING DATA GRID OPERATOR 14
3.1. INSTALLING DATA GRID OPERATOR ON RED HAT OPENSHIFT 14
3.2. INSTALLING DATA GRID OPERATOR WITH THE NATIVE CLI PLUGIN 15
3.3. INSTALLING DATA GRID OPERATOR WITH AN OPENSHIFT CLIENT 15

CHAPTER 4. CONFIGURING DATA GRID OPERATOR 18
4.1. CONFIGURE OPERATOR LOGGING 18
Logging format comparison 19

CHAPTER 5. CREATING DATA GRID CLUSTERS 20
5.1. INFINISPAN CUSTOM RESOURCE (CR) 20
5.2. CREATING DATA GRID CLUSTERS 20
5.3. VERIFYING DATA GRID CLUSTER VIEWS 21
Retrieving cluster view from logs 22
5.4. MODIFYING DATA GRID CLUSTERS 22
5.5. STOPPING AND STARTING DATA GRID CLUSTERS 23

CHAPTER 6. CONFIGURING DATA GRID CLUSTERS 24
6.1. APPLYING CUSTOM CONFIGURATION TO DATA GRID CLUSTERS 24
6.2. CUSTOM DATA GRID CONFIGURATION 25
6.2.1. Cache template 25
6.2.2. Logging configuration 27
6.3. SECURING CUSTOM DATA GRID CONFIGURATION 27

CHAPTER 7. UPGRADING DATA GRID CLUSTERS 30
7.1. TECHNOLOGY PREVIEW FEATURES 30
7.2. DATA GRID CLUSTER UPGRADES 30
Shutdown upgrades 30
InPlaceRolling upgrades 30
HotRodRolling upgrades 31
7.3. UPGRADING DATA GRID CLUSTERS WITH DOWNTIME 31
7.4. UPGRADING DATA GRID CLUSTERS USING INPLACE ROLLING STRATEGY 32
7.5. PERFORMING HOT ROD ROLLING UPGRADES FOR DATA GRID CLUSTERS 32
7.5.1. Recovering from a failed Hot Rod rolling upgrade 33

CHAPTER 8. SETTING UP DATA GRID SERVICES 34
8.1. SERVICE TYPES 34

8.2. CREATING DATA GRID SERVICE PODS 34
8.2.1. Data Grid service CR 34
8.3. ALLOCATING STORAGE RESOURCES 37
8.3.1. Persistent volume claims 38
8.4. ALLOCATING CPU AND MEMORY 38
8.5. SETTING JVM OPTIONS 39
8.6. CONFIGURING POD PROBES 40
8.7. CONFIGURING POD PRIORITY 41
8.8. CONFIGURING POD TERMINATION GRACE PERIOD 41
8.9. FIPS MODE FOR YOUR INFINISPAN CR 42
8.10. ADJUSTING LOG PATTERN 42
8.11. ADJUSTING LOG LEVELS 43
8.11.1. Logging reference 43
8.12. ADDING LABELS AND ANNOTATIONS TO DATA GRID RESOURCES 44
8.13. ADDING LABELS AND ANNOTATIONS WITH ENVIRONMENT VARIABLES 45
8.14. ADDING ENVIRONMENT VARIABLES TO DATA GRID PODS 46
8.15. DEFINING ENVIRONMENT VARIABLES IN THE DATA GRID OPERATOR SUBSCRIPTION 47

CHAPTER 9. CONFIGURING AUTHENTICATION 49
9.1. DEFAULT CREDENTIALS 49
9.2. RETRIEVING CREDENTIALS 49
9.3. ADDING CUSTOM USER CREDENTIALS 49
9.4. CHANGING THE OPERATOR PASSWORD 50
9.5. DISABLING USER AUTHENTICATION 50

CHAPTER 10. CONFIGURING CLIENT CERTIFICATE AUTHENTICATION 51
10.1. CLIENT CERTIFICATE AUTHENTICATION 51
10.2. ENABLING CLIENT CERTIFICATE AUTHENTICATION 51
10.3. PROVIDING CLIENT TRUSTSTORES 52
10.4. PROVIDING CLIENT CERTIFICATES 53

CHAPTER 11. CONFIGURING ENCRYPTION 54
11.1. ENCRYPTION WITH RED HAT OPENSHIFT SERVICE CERTIFICATES 54
11.2. RETRIEVING TLS CERTIFICATES 54
11.3. DISABLING ENCRYPTION 55
11.4. USING CUSTOM TLS CERTIFICATES 55
11.4.1. Custom encryption secrets 56

CHAPTER 12. CONFIGURING USER ROLES AND PERMISSIONS 57
12.1. ENABLING SECURITY AUTHORIZATION 57
12.2. USER ROLES AND PERMISSIONS 57
Data Grid Operator credentials 58
12.3. ASSIGNING ROLES AND PERMISSIONS TO USERS 58
12.4. ADDING CUSTOM ROLES AND PERMISSIONS 58

CHAPTER 13. CONFIGURING NETWORK ACCESS TO DATA GRID 60
13.1. GETTING THE SERVICE FOR INTERNAL CONNECTIONS 60
13.2. EXPOSING DATA GRID THROUGH A LOADBALANCER SERVICE 60
13.3. EXPOSING DATA GRID THROUGH A NODEPORT SERVICE 61
13.4. EXPOSING DATA GRID THROUGH A ROUTE 61
13.5. NETWORK SERVICES 62

CHAPTER 14. SETTING UP CROSS-SITE REPLICATION 64
14.1. CROSS-SITE REPLICATION EXPOSE TYPES 64
14.2. MANAGED CROSS-SITE REPLICATION 65

14.2.1. Creating service account tokens for managed cross-site connections 65
14.2.2. Exchanging service account tokens 66
14.2.3. Exchanging bound service account tokens 67
Deleting expired tokens 67
14.2.4. Configuring managed cross-site connections 68
14.3. MANUALLY CONFIGURING CROSS-SITE CONNECTIONS 70
14.4. ALLOCATING CPU AND MEMORY FOR GOSSIP ROUTER POD 73
14.5. DISABLING LOCAL GOSSIP ROUTER AND SERVICE 73
14.6. RESOURCES FOR CONFIGURING CROSS-SITE REPLICATION 75
Managed cross-site connections 76
Manual cross-site connections 77
14.7. SECURING CROSS-SITE CONNECTIONS 77
14.7.1. Resources for configuring cross-site encryption 79
14.7.2. Cross-site encryption secrets 80
14.8. CONFIGURING SITES IN THE SAME OPENSHIFT CLUSTER 81

CHAPTER 15. MONITORING DATA GRID SERVICES 83
15.1. CREATING A PROMETHEUS SERVICE MONITOR 83
15.1.1. Disabling the Prometheus service monitor 84
15.1.2. Configuring Service Monitor Target Labels 84
15.2. ENABLING JMX REMOTE PORTS FOR DATA GRID CLUSTERS 84
15.3. SETTING UP JFR RECORDINGS WITH CRYOSTAT 85

CHAPTER 16. GUARANTEEING AVAILABILITY WITH ANTI-AFFINITY 87
16.1. ANTI-AFFINITY STRATEGIES 87
Fault tolerance 87
16.2. CONFIGURING ANTI-AFFINITY 88
16.2.1. Anti-affinity strategy configurations 88
Schedule pods on different OpenShift nodes 88
Requiring different nodes 88
Schedule pods across multiple OpenShift zones 89
Requiring multiple zones 89

CHAPTER 17. AUTO SCALING 90
17.1. CONFIGURING HORIZONTALPODAUTOSCALER 90

CHAPTER 18. CREATING CACHES WITH DATA GRID OPERATOR 92
18.1. DATA GRID CACHES 92
Cache CRs 92
18.2. CREATING CACHES WITH THE CACHE CR 92
Cache CR examples 93
18.3. UPDATING CACHES WITH THE CACHE CR 93
18.4. ADDING PERSISTENT CACHE STORES 94

CHAPTER 19. RUNNING BATCH OPERATIONS 95
19.1. RUNNING INLINE BATCH OPERATIONS 95
19.2. CREATING CONFIGMAPS FOR BATCH OPERATIONS 95
19.3. RUNNING BATCH OPERATIONS WITH CONFIGMAPS 96
19.4. BATCH STATUS MESSAGES 97
19.5. EXAMPLE BATCH OPERATIONS 98
19.5.1. Caches 98
19.5.2. Counters 98
19.5.3. Protobuf schema 98
19.5.4. Tasks 99

CHAPTER 20. BACKING UP AND RESTORING DATA GRID CLUSTERS 100
20.1. BACKUP AND RESTORE CRS 100
20.2. BACKING UP DATA GRID CLUSTERS 100
20.3. RESTORING DATA GRID CLUSTERS 102
20.4. BACKUP AND RESTORE STATUS 103
20.4.1. Handling failed backup and restore operations 104

CHAPTER 21. DEPLOYING CUSTOM CODE TO DATA GRID 105
21.1. COPYING CODE ARTIFACTS TO DATA GRID CLUSTERS 105
21.2. DOWNLOADING CODE ARTIFACTS 107

CHAPTER 22. ESTABLISHING REMOTE CLIENT CONNECTIONS 109
22.1. CLIENT CONNECTION DETAILS 109
22.2. CONNECTING TO DATA GRID CLUSTERS WITH REMOTE SHELLS 109
22.3. ACCESSING DATA GRID CONSOLE 110
22.4. HOT ROD CLIENTS 111
Client intelligence 111
22.4.1. Hot Rod client configuration API 111
On OpenShift 111
Outside OpenShift 112
22.4.2. Configuring Hot Rod clients for certificate authentication 113
22.4.3. Creating caches from Hot Rod clients 114
Programmatically creating caches 114
Using Hot Rod client properties 115
22.5. ACCESSING THE REST API 115

RED HAT DATA GRID

Data Grid is a high-performance, distributed in-memory data store.

Schemaless data structure

Flexibility to store different objects as key-value pairs.

Grid-based data storage

Designed to distribute and replicate data across clusters.

Elastic scaling

Dynamically adjust the number of nodes to meet demand without service disruption.

Data interoperability

Store, retrieve, and query data in the grid from different endpoints.

DATA GRID DOCUMENTATION

Documentation for Data Grid is available on the Red Hat customer portal.

- Data Grid 8.6 Documentation
- Data Grid 8.6 Component Details
- Supported Configurations for Data Grid 8.6
- Data Grid 8 Feature Support
- Data Grid Deprecated Features and Functionality

DATA GRID DOWNLOADS

Access the Data Grid Software Downloads on the Red Hat customer portal.



NOTE

You must have a Red Hat account to access and download Data Grid software.

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see our CTO Chris Wright's message.

CHAPTER 1. DATA GRID OPERATOR

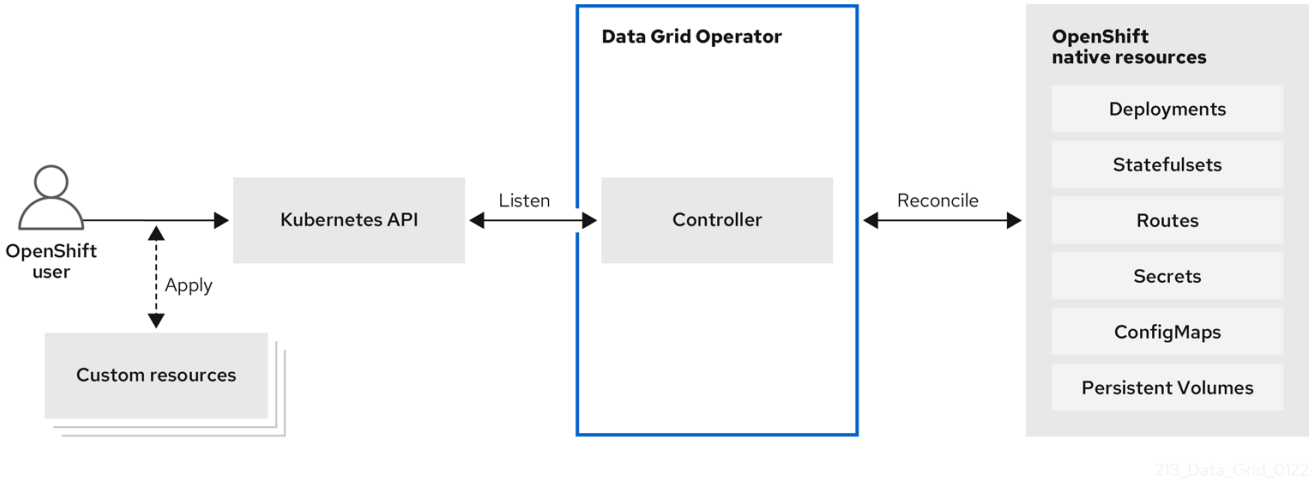
Data Grid Operator provides operational intelligence and reduces management complexity for deploying Data Grid on Kubernetes and Red Hat OpenShift.

1.1. DATA GRID OPERATOR DEPLOYMENTS

When you install Data Grid Operator, it extends the Kubernetes API with Custom Resource Definitions (CRDs) for deploying and managing Data Grid clusters on Red Hat OpenShift.

To interact with Data Grid Operator, OpenShift users apply Custom Resources (CRs) through the OpenShift Web Console or **oc** client. Data Grid Operator listens for **Infinispan** CRs and automatically provisions native resources, such as StatefulSets and Secrets, that your Data Grid deployment requires. Data Grid Operator also configures Data Grid services according to the specifications in **Infinispan** CRs, including the number of pods for the cluster and backup locations for cross-site replication.

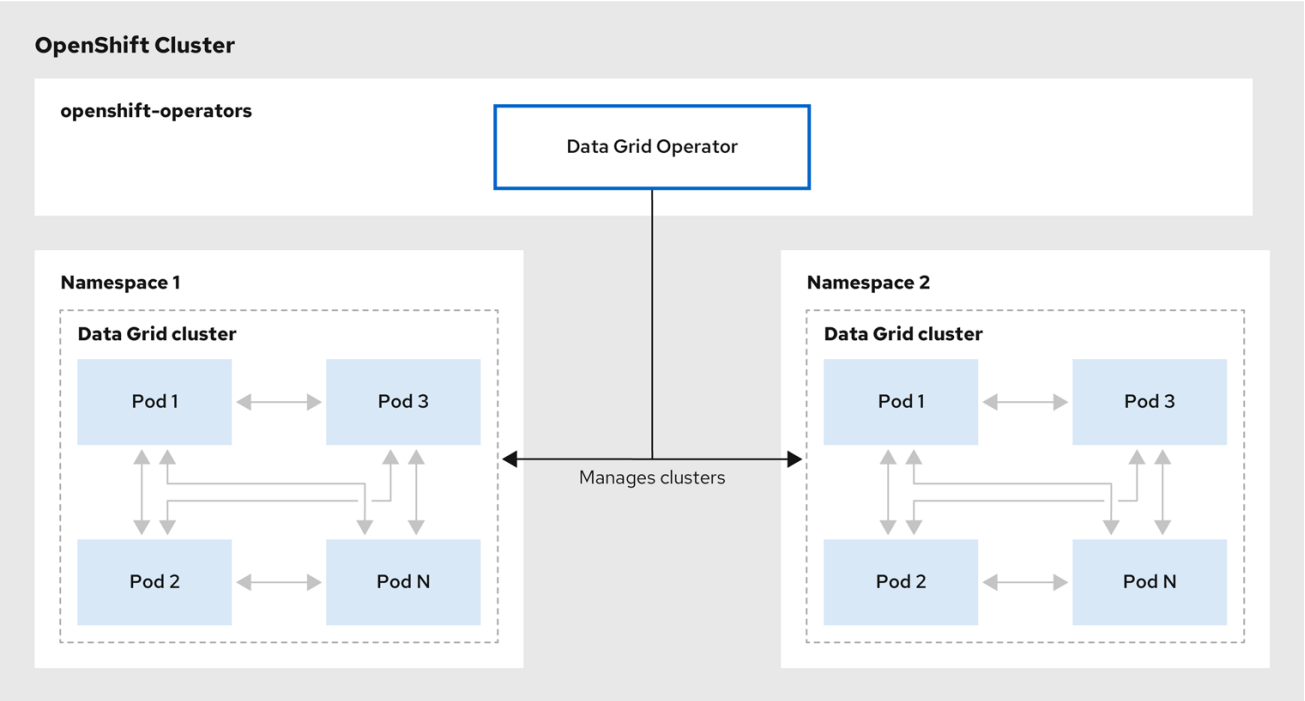
Figure 1.1. Custom resources



1.2. CLUSTER MANAGEMENT

A single Data Grid Operator installation can manage multiple clusters with different Data Grid versions in separate namespaces. Each time a user applies CRs to modify the deployment, Data Grid Operator applies the changes globally to all Data Grid clusters.

Figure 1.2. Operator-managed clusters



1.3. RESOURCE RECONCILIATION

Data Grid Operator reconciles custom resources such as the **Cache** CR with resources on your Data Grid cluster.

Bidirectional reconciliation synchronizes your CRs with changes that you make to Data Grid resources through the Data Grid Console, command line interface (CLI), or other client application and vice versa. For example if you create a cache through the Data Grid Console then Data Grid Operator adds a declarative Kubernetes representation.

To perform reconciliation Data Grid Operator creates a **listener** pod for each Data Grid cluster that detects modifications for **Infinispan** resources.

Notes about reconciliation

- When you create a cache through the Data Grid Console, CLI, or other client application, Data Grid Operator creates a corresponding **Cache** CR with a unique name that conforms to the Kubernetes naming policy.
- Declarative Kubernetes representations of Data Grid resources that Data Grid Operator creates with the **listener** pod are linked to **Infinispan** CRs.
  Deleting **Infinispan** CRs removes any associated resource declarations.

CHAPTER 2. INSTALLING THE NATIVE DATA GRID CLI AS A CLIENT PLUGIN

Data Grid provides a command line interface (CLI) compiled to a native executable that you can install as a plugin for **oc** clients. You can then use your **oc** client to:

- Create Data Grid Operator subscriptions and remove Data Grid Operator installations.
- Set up Data Grid clusters and configure services.
- Work with Data Grid resources via remote shells.

2.1. INSTALLING THE NATIVE DATA GRID CLI PLUGIN

Install the native Data Grid Command Line Interface (CLI) as a plugin for **oc** clients.

Prerequisites

- Have an **oc** client.
- Download the native Data Grid CLI distribution from the Data Grid software downloads.

Procedure

1. Extract the **.zip** archive for the native Data Grid CLI distribution.
2. Copy the native executable, or create a hard link, to a file named "kubectl-infinispan", for example:

```
cp redhat-datagrid-cli kubectl-infinispan
```

3. Add **kubectl-infinispan** to your **PATH**.
4. Verify that the CLI is installed.

```
oc plugin list

The following compatible plugins are available:
/path/to/kubectl-infinispan
```

5. Use the **infinispan --help** command to view available commands.

```
oc infinispan --help
```

Additional resources

- Extending the OpenShift CLI with plug-ins

2.2. KUBECTL-INFINISPAN COMMAND REFERENCE

This topic provides some details about the **kubectl-infinispan** plugin for clients.

TIP

Use the **--help** argument to view the complete list of available options and descriptions for each command.

For example, **oc infinispan create cluster --help** prints all command options for creating Data Grid clusters.

Command	Description
oc infinispan install	Creates Data Grid Operator subscriptions and installs into the global namespace by default.
oc infinispan create cluster	Creates Data Grid clusters.
oc infinispan get clusters	Displays running Data Grid clusters.
oc infinispan shell	Starts an interactive remote shell session on a Data Grid cluster.
oc infinispan delete cluster	Removes Data Grid clusters.
oc infinispan uninstall	Removes Data Grid Operator installations and all managed resources.

CHAPTER 3. INSTALLING DATA GRID OPERATOR

Install Data Grid Operator into a OpenShift namespace to create and manage Data Grid clusters.



NOTE

Because Custom Resource Definitions (CRDs) are deployed cluster-wide, you cannot run multiple versions of the Infinispan Operator on the same cluster.

To avoid issues, deploy the Operator cluster-wide and use the **spec.version** field in your Infinispan custom resources (CRs) to control upgrades of Infinispan server versions.

For more details, see https://access.redhat.com/solutions/7020126

3.1. INSTALLING DATA GRID OPERATOR ON RED HAT OPENSHIFT

Create subscriptions to Data Grid Operator on OpenShift so you can install different Data Grid versions and receive automatic updates.

Automatic updates apply to Data Grid Operator first and then for each Data Grid node. Data Grid Operator updates clusters one node at a time, gracefully shutting down each node and then bringing it back online with the updated version before going on to the next node.

Prerequisites

- Access to **OperatorHub** running on OpenShift. Some OpenShift environments, such as OpenShift Container Platform, can require administrator credentials.
- Have an OpenShift project for Data Grid Operator if you plan to install it into a specific namespace.

Procedure

1. Log in to the OpenShift Web Console.
2. Navigate to **OperatorHub**.
3. Find and select Data Grid Operator.
4. Select **Install** and continue to **Create Operator Subscription**.
5. Specify options for your subscription.

   Installation Mode

   You can install Data Grid Operator into a **Specific** namespace or **All** namespaces.

   Update Channel

   Get updates for Data Grid Operator 8.6.x.

   Approval Strategies

   Automatically install updates from the 8.6.x channel or require approval before installation.
6. Select **Subscribe** to install Data Grid Operator.
7. Navigate to **Installed Operators** to verify the Data Grid Operator installation.

3.2. INSTALLING DATA GRID OPERATOR WITH THE NATIVE CLI PLUGIN

Install Data Grid Operator with the native Data Grid CLI plugin, **kubectl-infinispan**.

Prerequisites

- Have **kubectl-infinispan** on your **PATH**.

Procedure

1. Run the **oc infinispan install** command to create Data Grid Operator subscriptions, for example:

```
oc infinispan install --channel=8.6.x
                      --source=redhat-operators
                      --source-namespace=openshift-marketplace
```

2. Verify the installation.

```
oc get pods -n openshift-operators | grep infinispan-operator
NAME                              READY   STATUS
infinispan-operator-<id>            1/1     Running
```

TIP

Use **oc infinispan install --help** for command options and descriptions.

3.3. INSTALLING DATA GRID OPERATOR WITH AN OPENSHIFT CLIENT

You can use the **oc** client to create Data Grid Operator subscriptions as an alternative to installing through the **OperatorHub** or with the native Data Grid CLI.

Prerequisites

- Have an **oc** client.

Procedure

1. Set up projects.
   a. Create a project for Data Grid Operator.
   b. If you want Data Grid Operator to control a specific Data Grid cluster only, create a project for that cluster.

```
oc new-project ${INSTALL_NAMESPACE} 1
oc new-project ${WATCH_NAMESPACE} 2
```

1 Creates a project into which you install Data Grid Operator.

2 Optionally creates a project for a specific Data Grid cluster if you do not want Data Grid Operator to watch all projects.

2. Create an **OperatorGroup** resource.

Control all Data Grid clusters

```
oc apply -f - << EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
 name: datagrid
 namespace: ${INSTALL_NAMESPACE}
EOF
```

Control a specific Data Grid cluster

```
oc apply -f - << EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
 name: datagrid
 namespace: ${INSTALL_NAMESPACE}
spec:
 targetNamespaces:
 - ${WATCH_NAMESPACE}
EOF
```

3. Create a subscription for Data Grid Operator.

```
oc apply -f - << EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
 name: datagrid-operator
 namespace: ${INSTALL_NAMESPACE}
spec:
 channel: 8.6.x
 installPlanApproval: Automatic
 name: datagrid
 source: redhat-operators
 sourceNamespace: openshift-marketplace
EOF
```



NOTE

If you want to manually approve updates from the 8.6.x channel, change the value of the **spec.installPlanApproval** field to **Manual**.

4. Verify the installation.

```
oc get pods -n ${INSTALL_NAMESPACE}
NAME                                READY   STATUS
```

```
infinispan-operator-<id>          1/1     Running
```

CHAPTER 4. CONFIGURING DATA GRID OPERATOR

4.1. CONFIGURE OPERATOR LOGGING

Configure the Data Grid Operator to output logs in JSON format for improved machine processing and log aggregation. You can also adjust the logging level to increase verbosity for debugging reconciliation issues.

INFINISPAN_OPERATOR_LOG_FORMAT variable

Specifies the output format.

- **console** (default): Human-readable text.
- **json**: Structured JSON format for log aggregators.

INFINISPAN_OPERATOR_LOG_LEVEL variable

Sets the verbosity of logs.

- Valid values are **debug**, **info** (default), **warn**, or **error**.

TIP

In production environments, it is highly recommended to set **INFINISPAN_OPERATOR_LOG_FORMAT** to **json**. This ensures logs are easily indexed by monitoring tools like Splunk, Elasticsearch, or Datadog.

Procedure

1. Open your Data Grid Operator Subscription for editing
2. Add the logging environment variables under the spec.config.env field:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: datagrid
  namespace: openshift-operators
spec:
  channel: 8.6.x
  installPlanApproval: Automatic
  name: datagrid
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  config:
    env:
    - name: INFINISPAN_OPERATOR_LOG_FORMAT
      value: json
    - name: INFINISPAN_OPERATOR_LOG_LEVEL
      value: warn
```

3. Apply the changes.OLM will automatically restart the Operator pod to apply the new configuration.

4. Retrieve logs from Operator pod as required.

```
oc logs -f deployment/infinispan-operator-controller-manager
```

Logging format comparison

Console

```
2026-04-14T09:28:48.546Z INFO controllers.Infinispan Triggering StatefulSet Rolling upgrade
{"infinispan": "infinispan/infinispan"}
```

JSON

```
{"level":"info","ts":"2026-04-14T09:33:53.960Z","logger":"controllers.Infinispan","msg":"Triggering
StatefulSet Rolling upgrade","infinispan":"infinispan/infinispan"}
```

CHAPTER 5. CREATING DATA GRID CLUSTERS

Create Data Grid clusters running on OpenShift with the **Infinispan** CR or with the native Data Grid CLI plugin for **oc** clients.

5.1. INFINISPAN CUSTOM RESOURCE (CR)

Data Grid Operator adds a new Custom Resource (CR) of type **Infinispan** that lets you handle Data Grid clusters as complex units on OpenShift.

Data Grid Operator listens for **Infinispan** Custom Resources (CR) that you use to instantiate and configure Data Grid clusters and manage OpenShift resources, such as StatefulSets and Services.

Infinispan CR

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  version: <Data Grid_version>
  service:
    type: DataGrid
```

Field	Description
apiVersion	Declares the version of the **Infinispan** API.
kind	Declares the **Infinispan** CR.
metadata.name	Specifies a name for your Data Grid cluster.
spec.replicas	Specifies the number of pods in your Data Grid cluster.
spec.service.type	Specifies the type of Data Grid service to create.
spec.version	Specifies the Data Grid Server version of your cluster.

5.2. CREATING DATA GRID CLUSTERS

Create Data Grid clusters with the native CLI plugin, **kubectl-infinispan**.

Prerequisites

- Install Data Grid Operator.
- Have **kubectl-infinispan** on your **PATH**.

Procedure

1. Run the **infinispan create cluster** command.
   For example, create a Data Grid cluster with two pods as follows:

```
oc infinispan create cluster --replicas=3 -Pservice.type=DataGrid infinispan
```

   TIP

   Add the **--version** argument to control the Data Grid version of your cluster. For example, **--version=8.6.0-1**. If you don't specify the version, Data Grid Operator creates cluster with the latest supported Data Grid version.

2. Watch Data Grid Operator create the Data Grid pods.

```
oc get pods -w
```

Next steps

After you create a Data Grid cluster, use the **oc** to apply changes to **Infinispan** CR and configure your Data Grid service.

You can also delete Data Grid clusters with **kubectl-infinispan** and re-create them as required.

```
oc infinispan delete cluster infinispan
```

Additional resources

- kubectl-infinispan command reference

5.3. VERIFYING DATA GRID CLUSTER VIEWS

Confirm that Data Grid pods have successfully formed clusters.

Prerequisites

- Create at least one Data Grid cluster.

Procedure

- Retrieve the **Infinispan** CR for Data Grid Operator.

```
oc get infinispan -o yaml
```

  The response indicates that Data Grid pods have received clustered views, as in the following example:

```
conditions:
  - message: 'View: [infinispan-0, infinispan-1]'
    status: "True"
    type: wellFormed
```

TIP

Do the following for automated scripts:

```
oc wait --for condition=wellFormed --timeout=240s infinispan/infinispan
```

Retrieving cluster view from logs

You can also get the cluster view from Data Grid logs as follows:

```
oc logs infinispan-0 | grep ISPN000094
```

```
INFO  [org.infinispan.CLUSTER] (MSC service thread 1-2) \
ISPN000094: Received new cluster view for channel infinispan: \
[infinispan-0|0] (1) [infinispan-0]

INFO  [org.infinispan.CLUSTER] (jgroups-3,infinispan-0) \
ISPN000094: Received new cluster view for channel infinispan: \
[infinispan-0|1] (2) [infinispan-0, infinispan-1]
```

5.4. MODIFYING DATA GRID CLUSTERS

Configure Data Grid clusters by providing Data Grid Operator with a custom **Infinispan** CR.

Prerequisites

- Install Data Grid Operator.
- Create at least one Data Grid cluster.
- Have an **oc** client.

Procedure

1. Create a YAML file that defines your **Infinispan** CR.
   For example, create a **my_infinispan.yaml** file that changes the number of Data Grid pods to two:

```
cat > cr_minimal.yaml<<EOF
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  version: <Data Grid_version>
  service:
    type: DataGrid
EOF
```

2. Apply your **Infinispan** CR.

```
oc apply -f my_infinispan.yaml
```

3. Watch Data Grid Operator scale the Data Grid pods.

```
oc get pods -w
```

5.5. STOPPING AND STARTING DATA GRID CLUSTERS

Stop and start Data Grid pods in a graceful, ordered fashion to correctly preserve cluster state.

Clusters of Data Grid service pods must restart with the same number of pods that existed before shutdown. This allows Data Grid to restore the distribution of data across the cluster. After Data Grid Operator fully restarts the cluster you can safely add and remove pods.

Procedure

1. Change the **spec.replicas** field to **0** to stop the Data Grid cluster.

```
spec:
  replicas: 0
```

2. Ensure you have the correct number of pods before you restart the cluster.

```
oc get infinispan infinispan -o=jsonpath='{.status.replicasWantedAtRestart}'
```

3. Change the **spec.replicas** field to the same number of pods to restart the Data Grid cluster.

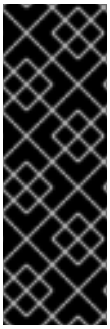
```
spec:
  replicas: 6
```

CHAPTER 6. CONFIGURING DATA GRID CLUSTERS

Apply custom Data Grid configuration to clusters that Data Grid Operator manages.

6.1. APPLYING CUSTOM CONFIGURATION TO DATA GRID CLUSTERS

Add Data Grid configuration to a **ConfigMap** and make it available to Data Grid Operator. Data Grid Operator can then apply the custom configuration to your Data Grid cluster.



IMPORTANT

Data Grid Operator applies default configuration on top of your custom configuration to ensure it can continue to manage your Data Grid clusters.

Be careful when applying custom configuration outside the **cache-container** element or field. You can apply custom configuration to underlying Data Grid Server mechanisms such as endpoints, security realms, and cluster transport. Changing this configuration can result in error and result in service downtime for your Data Grid deployment.

TIP

Use the Data Grid Helm chart to deploy clusters of fully configurable Data Grid Server instances on OpenShift.

Prerequisites

- Have valid Data Grid configuration in XML, YAML, or JSON format.

Procedure

1. Add Data Grid configuration to a **infinispan-config.[xml|yaml|json]** key in the **data** field of your **ConfigMap**.

   XML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-config
  namespace: rhdg-namespace
data:
  infinispan-config.xml: >
    <infinispan>
      <!-- Custom configuration goes here. -->
    </infinispan>
```

   YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-config
  namespace: rhdg-namespace
```

```
data:
  infinispan-config.yaml: >
    infinispan:
      # Custom configuration goes here.
```

JSON

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-config
  namespace: rhdg-namespace
data:
  infinispan-config.json: >
    {
      "infinispan": {
      }
    }
```

2. Create the **ConfigMap** from your YAML file.

```
oc apply -f cluster-config.yaml
```

3. Specify the name of the **ConfigMap** with the **spec.configMapName** field in your **Infinispan** CR and then apply the changes.

```
spec:
  configMapName: "cluster-config"
```

Next steps

If your cluster is already running Data Grid Operator restarts it to apply the configuration. Each time you modify the Data Grid configuration in the **ConfigMap**, Data Grid Operator detects the updates and restarts the cluster to apply the changes.

Additional resources

- Data Grid Helm chart

6.2. CUSTOM DATA GRID CONFIGURATION

You can add Data Grid configuration to a **ConfigMap** in XML, YAML, or JSON format.

6.2.1. Cache template

XML

```
<infinispan>
  <cache-container>
    <distributed-cache-configuration name="base-template">
      <expiration lifespan="5000"/>
    </distributed-cache-configuration>
    <distributed-cache-configuration name="extended-template"
```

```
                                configuration="base-template">
      <encoding media-type="application/x-protostream"/>
      <expiration lifespan="10000"
                  max-idle="1000"/>
    </distributed-cache-configuration>
  </cache-container>
</infinispan>
```

YAML

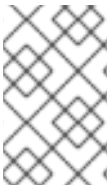
```
infinispan:
  cacheContainer:
    caches:
      base-template:
        distributedCacheConfiguration:
          expiration:
            lifespan: "5000"
      extended-template:
        distributedCacheConfiguration:
          configuration: "base-template"
          encoding:
            mediaType: "application/x-protostream"
          expiration:
            lifespan: "10000"
            maxIdle: "1000"
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
      "caches" : {
        "base-template" : {
          "distributed-cache-configuration" : {
            "expiration" : {
              "lifespan" : "5000"
            }
          }
        },
        "extended-template" : {
          "distributed-cache-configuration" : {
            "configuration" : "base-template",
            "encoding": {
              "media-type": "application/x-protostream"
              },
            "expiration" : {
              "lifespan" : "10000",
              "max-idle" : "1000"
            }
          }
        }
      }
    }
  }
}
```

6.2.2. Logging configuration

You can also include Apache Log4j configuration in XML format as part of your **ConfigMap**.



NOTE

Use the **spec.logging.categories** field in your **Infinispan** CR to adjust logging levels for Data Grid clusters. Add Apache Log4j configuration only if you require advanced file-based logging capabilities.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: logging-config
  namespace: rhdg-namespace
data:
  infinispan-config.xml: >
    <infinispan>
      <!-- Add custom Data Grid configuration if required. -->
      <!-- You can provide either Data Grid configuration, logging configuration, or both. -->
    </infinispan>

  log4j.xml: >
    <?xml version="1.0" encoding="UTF-8"?>
    <Configuration name="ServerConfig" monitorInterval="60" shutdownHook="disable">
        <Appenders>
            <!-- Colored output on the console -->
            <Console name="STDOUT">
                <PatternLayout pattern="%d{HH:mm:ss,SSS} %-5p (%t) [%c] %m%throwable%n"/>
            </Console>
        </Appenders>

        <Loggers>
            <Root level="INFO">
                <AppenderRef ref="STDOUT" level="TRACE"/>
            </Root>
            <Logger name="org.infinispan" level="TRACE"/>
        </Loggers>
    </Configuration>
```

Additional resources

- Custom Data Grid configuration

6.3. SECURING CUSTOM DATA GRID CONFIGURATION

Securely define and store custom Data Grid Server configuration. To protect sensitive text strings such as passwords, add the entries in a credential store rather than directly in the Data Grid Server configuration.

Prerequisites

- Have a valid Data Grid configuration in XML, YAML, or JSON format.

Procedure

1. Create a **CredentialStore Secret** file.
2. Use the **data** field to specify the credentials and its aliases.

   user-secret.yaml

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: user-secret
   type: Opaque
   data:
     postgres_cred: sensitive-value
     mysql_cred: sensitive-value2
   ```

3. Apply your Secret file.

   ```
   oc apply -f user-secret.yaml
   ```

4. Open the **Infinispan** CR for editing.
5. In the **spec.security.credentialStoreSecretName** field, specify the name of the credential store secret.

   Infinispan CR

   ```
   spec:
     security:
       credentialStoreSecretName: user-secret
   ```

6. Apply the changes.
7. Open your Data Grid Server configuration for editing.
8. Add a **credential-reference** to your configuration where required.
   a. Specify the **credentials** as the name of the **store**.
   b. Specify the **alias** attribute as one of the keys defined in your credential secret.

      Data Grid.xml

      ```
      <data-source name="postgres" jndi-name="jdbc/postgres">
         <connection-factory driver="org.postgresql.Driver"
                             username="dbuser"
                             url="${org.infinispan.server.test.postgres.jdbcUrl}">
            <!-- Credential 'postgres_cred' defined in the 'user-secret' Secret is used here as the
      database password -->
            <credential-reference store="credentials" alias="postgres_cred"/>
         </connection-factory>
         <connection-pool />
      </data-source>
      ```

Additional resources

- Credential store references

CHAPTER 7. UPGRADING DATA GRID CLUSTERS

Data Grid Operator lets you upgrade Data Grid clusters from one version to another without downtime or data loss.



IMPORTANT

Hot Rod rolling upgrades are available as a technology preview feature.

7.1. TECHNOLOGY PREVIEW FEATURES

Technology preview features or capabilities are not supported with Red Hat production service-level agreements (SLAs) and might not be functionally complete.

Red Hat does not recommend using technology preview features or capabilities for production. These features provide early access to upcoming product features, which enables you to test functionality and provide feedback during the development process.

For more information, see Red Hat Technology Preview Features Support Scope .

7.2. DATA GRID CLUSTER UPGRADES

The **spec.upgrades.type** field controls how Data Grid Operator upgrades your Data Grid cluster when new versions become available. There are three types of cluster upgrade:

Shutdown

Upgrades Data Grid clusters, with service downtime, by bringing down the original cluster and bringing up a new one in its place. This is the default upgrade type.

InPlaceRolling

Upgrades Data Grid cluster pods to new patch releases individually, using a StatefulSet RollingUpdate.

HotRodRolling

Upgrades Data Grid versions, without service downtime, by deploying a new cluster in parallel to the original and transitioning all traffic to it.



IMPORTANT

The Operator does not automatically upgrade clusters to a new Data Grid server version. In order to trigger an upgrade, you must explicitly change the **spec.version** element to a new version.

Shutdown upgrades

To perform a shutdown upgrade, Data Grid Operator does the following:

1. Gracefully shuts down the existing cluster.
2. Removes the existing cluster.
3. Creates a new cluster with the target version.

InPlaceRolling upgrades

To perform a rolling upgrade, Data Grid Operator does the following:

1. Checks if the Data Grid version supports rolling upgrades.
2. Checks whether the current version and target version have matching major and minor versions.
3. If the conditions above are met, the Operator will upgrade server pods one by one using a StatefulSet RollingUpdate.



IMPORTANT

=== When upgrading between different major or minor versions of the server, the "Shutdown" or "HotRodRolling" upgrade type is required. This must be configured in the CR spec at the the same time as the new version. ===

HotRodRolling upgrades

To perform a Hot Rod rolling upgrade, Data Grid Operator does the following:

1. Creates a new Data Grid cluster with the target version that runs alongside your existing cluster.
2. Creates a remote cache store to transfer data from the existing cluster to the new cluster.
3. Redirects all clients to the new cluster.
4. Removes the existing cluster when all data and client connections are transferred to the new cluster.



IMPORTANT

You should not perform Hot Rod rolling upgrades with caches that enable passivation with persistent cache stores. In the event that the upgrade does not complete successfully, passivation can result in data loss when Data Grid Operator rolls back the target cluster.

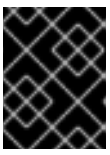
If your cache configuration enables passivation you should perform a shutdown upgrade.

7.3. UPGRADING DATA GRID CLUSTERS WITH DOWNTIME

Upgrading Data Grid clusters with downtime results in service disruption but does not require any additional capacity.

Prerequisites

- If required, configure a persistent cache store to preserve your data during the upgrade.



IMPORTANT

At the start of the upgrade process Data Grid Operator shuts down your existing cluster. This results in data loss if you do not configure a persistent cache store.

Procedure

1. Specify the Data Grid version number in the **spec.version** field.

2. Ensure that **Shutdown** is set as the value for the **spec.upgrades.type** field, which is the default.

```
spec:
  version: 8.6.0-1
  upgrades:
    type: Shutdown
```

3. Apply your changes, if necessary.

7.4. UPGRADING DATA GRID CLUSTERS USING INPLACE ROLLING STRATEGY

Upgrading Data Grid clusters using InPlace rolling strategy with no service downtime.

Prerequisites

- Upgrading to a new Data Grid server patch release

Procedure

1. Specify the Data Grid version number in the **spec.version** field.
2. Ensure that **InPlaceRolling** is set as the value for the **spec.upgrades.type** field.

```
spec:
  version: 8.6.0-1
  upgrades:
    type: InPlaceRolling
```

3. Apply your changes, if necessary.

7.5. PERFORMING HOT ROD ROLLING UPGRADES FOR DATA GRID CLUSTERS

Performing Hot Rod rolling upgrades lets you move to a new Data Grid version without service disruption. However, this upgrade type requires additional capacity and temporarily results in two Data Grid clusters with different versions running concurrently.

Prerequisite

- The Data Grid Operator version you have installed supports the Data Grid target version.

Procedure

1. Specify the Data Grid version number in the **spec.version** field.
2. Specify **HotRodRolling** as the value for the **spec.upgrades.type** field.

```
spec:
  version: 8.6.0-1
  upgrades:
    type: HotRodRolling
```

3. Apply your changes.

7.5.1. Recovering from a failed Hot Rod rolling upgrade

You can roll back a failed Hot Rod rolling upgrade to the previous version if the original cluster is still present.

Prerequisites

- Hot Rod rolling upgrade is in progress and the initial Data Grid cluster is present.

Procedure

1. Ensure the Hot Rod rolling upgrade is in progress.

```
oc get infinispan <cr_name> -o yaml
```

   The **status.hotRodRollingUpgradeStatus** field must be present.

2. Update **spec.version** field of your **Infinispan CR** to the original cluster version defined in the **status.hotRodRollingUpgradeStatus**.
   Data Grid Operator deletes the newly created cluster.

CHAPTER 8. SETTING UP DATA GRID SERVICES

Use Data Grid Operator to create clusters of Data Grid service pods.

8.1. SERVICE TYPES

Services are stateful applications, based on the Data Grid Server image, that provide flexible and robust in-memory data storage. Data Grid operator supports only **DataGrid** service type which deploys Data Grid clusters with full configuration and capabilities. **Cache** service type is no longer supported.

DataGrid` service type for clusters lets you:

- Back up data across global clusters with cross-site replication.
- Create caches with any valid configuration.
- Add file-based cache stores to save data in a persistent volume.
- Query values across caches using the Data Grid Query API.
- Use advanced Data Grid features and capabilities.

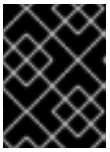
8.2. CREATING DATA GRID SERVICE PODS

To use custom cache definitions along with Data Grid capabilities such as cross-site replication, create clusters of Data Grid service pods.

Procedure

1. Create an **Infinispan** CR that sets **spec.service.type: DataGrid** and configures any other Data Grid service resources.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  version: <Data Grid_version>
  service:
    type: DataGrid
```



IMPORTANT

You cannot change the **spec.service.type** field after you create pods. To change the service type, you must delete the existing pods and create new ones.

2. Apply your **Infinispan** CR to create the cluster.

8.2.1. Data Grid service CR

This topic describes the **Infinispan** CR for Data Grid service pods.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
  annotations:
    infinispan.org/monitoring: 'true'
spec:
  replicas: 6
  version: 8.6.0-1
  upgrades:
    type: Shutdown
  service:
    type: DataGrid
    container:
      storage: 2Gi
      # The ephemeralStorage and storageClassName fields are mutually exclusive.
      ephemeralStorage: false
      storageClassName: my-storage-class
    sites:
      local:
      name: azure
      expose:
        type: LoadBalancer
      locations:
      - name: azure
        url: openshift://api.azure.host:6443
        secretName: azure-token
      - name: aws
        clusterName: infinispan
        namespace: rhdg-namespace
        url: openshift://api.aws.host:6443
        secretName: aws-token
  security:
    endpointSecretName: endpoint-identities
    endpointEncryption:
        type: Secret
        certSecretName: tls-secret
  container:
    extraJvmOpts: "-XX:NativeMemoryTracking=summary"
    cpu: "2000m:1000m"
    memory: "2Gi:1Gi"
  logging:
    categories:
      org.infinispan: debug
      org.jgroups: debug
      org.jgroups.protocols.TCP: error
      org.jgroups.protocols.relay.RELAY2: error
  expose:
    type: LoadBalancer
  configMapName: "my-cluster-config"
  configListener:
    enabled: true
  scheduling:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
```

```
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: infinispan-pod
              clusterName: infinispan
              infinispan_cr: infinispan
          topologyKey: "kubernetes.io/hostname"
```

Field	Description
metadata.name	Names your Data Grid cluster.
metadata.annotations.infinispan.org/monitoring	Automatically creates a **ServiceMonitor** for your cluster.
spec.replicas	Specifies the number of pods in your cluster.
spec.version	Specifies the Data Grid Server version of your cluster.
spec.upgrades.type	Controls how Data Grid Operator upgrades your Data Grid cluster when new versions become available.
spec.service.type	Configures the type Data Grid service. A value of **DataGrid** creates a cluster with Data Grid service pods.
spec.service.container	Configures the storage resources for Data Grid service pods.
spec.service.sites	Configures cross-site replication.
spec.security.endpointSecretName	Specifies an authentication secret that contains Data Grid user credentials.
spec.security.endpointEncryption	Specifies TLS certificates and keystores to encrypt client connections.
spec.container	Specifies JVM, CPU, and memory resources for Data Grid pods.
spec.logging	Configures Data Grid logging categories.
spec.expose	Controls how Data Grid endpoints are exposed on the network.

Field	Description
spec.configMapName	Specifies a **ConfigMap** that contains Data Grid configuration.
spec.configListener.enabled	Creates a **listener** pod in each Data Grid cluster that allows Data Grid Operator to reconcile server-side modifications with Data Grid resources such as the **Cache** CR. The **listener** pod consumes minimal resources and is enabled by default. Setting a value of **false** removes the **listener** pod and disables bi-directional reconciliation. You should do this only if you do not need declarative Kubernetes representations of Data Grid resources created through the Data Grid Console, CLI, or client applications.
spec.configListener.logging.level	Configures the logging level for the **ConfigListener** deployments. The default level is **info**. You can change it to **debug** or **error**.
spec.scheduling.affinity	Configures anti-affinity strategies that guarantee Data Grid availability.

8.3. ALLOCATING STORAGE RESOURCES

By default, Data Grid Operator allocates **1Gi** for the persistent volume claim. However you should adjust the amount of storage available to Data Grid service pods so that Data Grid can preserve cluster state during shutdown.

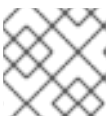


IMPORTANT

If available container storage is less than the amount of available memory, data loss can occur.

Procedure

1. Allocate storage resources with the **spec.service.container.storage** field.
2. Configure either the **ephemeralStorage** field or the **storageClassName** field as required.



NOTE

These fields are mutually exclusive. Add only one of them to your **Infinispan** CR.

3. Apply the changes.

Ephemeral storage

```
spec:
```

```
  service:
    type: DataGrid
    container:
      storage: 2Gi
      ephemeralStorage: true
```

Name of a StorageClass object

```
spec:
  service:
    type: DataGrid
    container:
      storage: 2Gi
      storageClassName: my-storage-class
```

Field	Description
spec.service.container.storage	Specifies the amount of storage for Data Grid service pods.
spec.service.container.ephemeralStorage	Defines whether storage is ephemeral or permanent. Set the value to **true** to use ephemeral storage, which means all data in storage is deleted when clusters shut down or restart. The default value is **false**, which means storage is permanent.
spec.service.container.storageClassName	Specifies the name of a **StorageClass** object to use for the persistent volume claim (PVC). If you include this field, you must specify an existing storage class as the value. If you do not include this field, the persistent volume claim uses the storage class that has the **storageclass.kubernetes.io/is-default-class** annotation set to **true**.

8.3.1. Persistent volume claims

Data Grid Operator creates a persistent volume claim (PVC) and mounts container storage at: **/opt/infinispan/server/data**

Caches

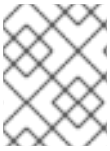
When you create caches, Data Grid permanently stores their configuration so your caches are available after cluster restarts.

Data

Use a file-based cache store, by adding the **<file-store/>** element to your Data Grid cache configuration, if you want Data Grid service pods to persist data during cluster shutdown.

8.4. ALLOCATING CPU AND MEMORY

Allocate CPU and memory resources to Data Grid pods with the **Infinispan** CR.



NOTE

Data Grid Operator requests **1Gi** of memory from the OpenShift scheduler when creating Data Grid pods. CPU requests are unbounded by default.

Procedure

1. Allocate the number of CPU units with the **spec.container.cpu** field.
2. Allocate the amount of memory, in bytes, with the **spec.container.memory** field.
   The **cpu** and **memory** fields have values in the format of **<limit>:<requests>**. For example, **cpu: "2000m:1000m"** limits pods to a maximum of **2000m** of CPU and requests **1000m** of CPU for each pod at startup. Specifying a single value sets both the limit and request.
3. Apply your **Infinispan** CR.
   If your cluster is running, Data Grid Operator restarts the Data Grid pods so changes take effect.

```
spec:
  container:
    cpu: "2000m:1000m"
    memory: "2Gi:1Gi"
```

8.5. SETTING JVM OPTIONS

Pass additional JVM options to Data Grid pods at startup.

Procedure

1. Configure JVM options with the **spec.container** filed in your **Infinispan** CR.
2. Apply your **Infinispan** CR.
   If your cluster is running, Data Grid Operator restarts the Data Grid pods so changes take effect.

JVM options

```
spec:
  container:
    extraJvmOpts: "-<option>=<value>"
    routerExtraJvmOpts: "-<option>=<value>"
    cliExtraJvmOpts: "-<option>=<value>"
```

Field	Description
spec.container.extraJvmOpts	Specifies additional JVM options for the Data Grid Server.

Field	Description
spec.container.routerExtraJvmOpts	Specifies additional JVM options for the Gossip router.
spec.container.cliExtraJvmOpts	Specifies additional JVM options for the Data Grid CLI.

8.6. CONFIGURING POD PROBES

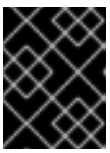
Optionally configure the values of the Liveness, Readiness and Startup probes used by Data Grid pods.

The Data Grid Operator automatically configures the probe values to sensible defaults. We only recommend providing your own values once you have determined that the default values do not match your requirements.

Procedure

1. Configure probe values using the **spec.service.container.*Probe** fields:

```
spec:
  service:
    container:
      readinessProbe:
        failureThreshold: 1
        initialDelaySeconds: 1
        periodSeconds: 1
        successThreshold: 1
        timeoutSeconds: 1
      livenessProbe:
        failureThreshold: 1
        initialDelaySeconds: 1
        periodSeconds: 1
        successThreshold: 1
        timeoutSeconds: 1
      startupProbe:
        failureThreshold: 1
        initialDelaySeconds: 1
        periodSeconds: 1
        successThreshold: 1
        timeoutSeconds: 1
```



IMPORTANT

If no value is specified for a given probe value, then the Data Grid Operator default is used.

2. Apply your **Infinispan** CR.
   If your cluster is running, Data Grid Operator restarts the Data Grid pods in order for the changes to take effect.

8.7. CONFIGURING POD PRIORITY

Create one or more priority classes to indicate the importance of a pod relative to other pods. Pods with higher priority are scheduled ahead of pods with lower priority, ensuring prioritization of pods running critical workloads, especially when resources become constrained.

Prerequisites

- Have **cluster-admin** access to OpenShift.

Procedure

1. Define a **PriorityClass** object by specifying its name and value.

   high-priority.yaml

   ```
   apiVersion: scheduling.k8s.io/v1
   kind: PriorityClass
   metadata:
     name: high-priority
   value: 1000000
   globalDefault: false
   description: "Use this priority class for high priority service pods only."
   ```

2. Create the priority class.

   ```
   oc create -f high-priority.yaml
   ```

3. Reference the priority class name in the pod configuration.

   Infinispan CR

   ```
   kind: Infinispan
   ...
   spec:
     scheduling:
       affinity:
         ...
       priorityClassName: "high-priority"
       ...
   ```

   You must reference an existing priority class name, otherwise the pod is rejected.

4. Apply the changes.

Additional resources

- Including pod priority in pod scheduling decisions

8.8. CONFIGURING POD TERMINATION GRACE PERIOD

Optionally configure the **TerminationGracePeriodSeconds** of the Data Grid pods.

The Data Grid Operator relies on the Kubernetes default **TerminationGracePeriodSeconds** unless a value is explicitly provided. We only recommend providing your own value once you have determined that the default value do not match your requirements.

Procedure

1. Configure probe values using the **spec.service.container.terminationGracePeriodSeconds** field:

```
spec:
  service:
    container:
      terminationGracePeriodSeconds: 60
```

2. Apply your **Infinispan** CR.
   If your cluster is running, Data Grid Operator restarts the Data Grid pods in order for the changes to take effect.

8.9. FIPS MODE FOR YOUR INFINISPAN CR

The Red Hat OpenShift Container Platform can use certain Federal Information Processing Standards (FIPS) components that ensure OpenShift clusters meet the requirements of a FIPS compliance audit.

If you enabled FIPS mode on your OpenShift cluster then the Data Grid Operator automatically enables FIPS mode for your **Infinispan** custom resource (CR).



IMPORTANT

Client certificate authentication is not currently supported with FIPS mode. Attempts to create **Infinispan** CR with **spec.security.endpointEncryption.clientCert** set to a value other than **None** will fail.

Additional resources

- Support for FIPS cryptography Red Hat OpenShift Container Platform

8.10. ADJUSTING LOG PATTERN

To customize the log display for Data Grid log traces, update the log pattern. If no custom pattern is set, the default format is: **%d{HH:mm:ss,SSS} %-5p (%t) [%c] %m%throwable%n**

Procedure

1. Configure Data Grid logging with the **spec.logging.pattern** field in your **Infinispan** CR.

```
spec:
  logging:
    pattern: %X{address} %X{user} [%d{dd/MMM/yyyy:HH:mm:ss Z}]
```

2. Apply the changes.
3. Retrieve logs from Data Grid pods as required.

```
oc logs -f $POD_NAME
```

8.11. ADJUSTING LOG LEVELS

Change levels for different Data Grid logging categories when you need to debug issues. You can also adjust log levels to reduce the number of messages for certain categories to minimize the use of container resources.

Procedure

1. Configure Data Grid logging with the **spec.logging.categories** field in your **Infinispan** CR.

```
spec:
  logging:
    categories:
      org.infinispan: debug
      org.jgroups: debug
```

2. Apply the changes.
3. Retrieve logs from Data Grid pods as required.

```
oc logs -f $POD_NAME
```

8.11.1. Logging reference

Find information about log categories and levels.

Table 8.1. Log categories

Root category	Description	Default level
org.infinispan	Data Grid messages	**info**
org.jgroups	Cluster transport messages	**info**

Table 8.2. Log levels

Log level	Description
trace	Provides detailed information about running state of applications. This is the most verbose log level.
debug	Indicates the progress of individual requests or activities.
info	Indicates overall progress of applications, including lifecycle events.

Log level	Description
warn	Indicates circumstances that can lead to error or degrade performance.
error	Indicates error conditions that might prevent operations or activities from being successful but do not prevent applications from running.

Garbage collection (GC) messages

Data Grid Operator does not log GC messages by default. You can direct GC messages to **stdout** with the following JVM options:

```
extraJvmOpts: "-Xlog:gc*:stdout:time,level,tags"
```

8.12. ADDING LABELS AND ANNOTATIONS TO DATA GRID RESOURCES

Attach key/value labels and annotations to pods and services that Data Grid Operator creates and manages. Labels help you identify relationships between objects to better organize and monitor Data Grid resources. Annotations are arbitrary non-identifying metadata for client applications or deployment and management tooling.



NOTE

Red Hat subscription labels are automatically applied to Data Grid resources.

Procedure

1. Open your **Infinispan** CR for editing.
2. Attach labels and annotations to Data Grid resources in the **metadata.annotations** section.
   - Define values for annotations directly in the **metadata.annotations** section.
   - Define values for labels with the **metadata.labels** field.
3. Apply your **Infinispan** CR.

Custom annotations

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  annotations:
    infinispan.org/targetAnnotations: service-annotation1, service-annotation2
    infinispan.org/podTargetAnnotations: pod-annotation1, pod-annotation2
    infinispan.org/routerAnnotations: router-annotation1, router-annotation2

    service-annotation1: value
    service-annotation2: value
```

```
    pod-annotation1: value
    pod-annotation2: value
    router-annotation1: value
    router-annotation2: value
```

Custom labels

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  annotations:
    infinispan.org/targetLabels: service-label1, service-label2
    infinispan.org/podTargetLabels: pod-label1, pod-label2
  labels:
    service-label1: value
    service-label2: value
    pod-label1: value
    pod-label2: value
    # The operator does not attach these labels to resources.
    my-label: my-value
    environment: development
```

8.13. ADDING LABELS AND ANNOTATIONS WITH ENVIRONMENT VARIABLES

Set environment variables for Data Grid Operator to add labels and annotations that automatically propagate to all Data Grid pods and services.

Procedure

Add labels and annotations to your Data Grid Operator subscription with the **spec.config.env** field in one of the following ways:

- Use the **oc edit subscription** command.

```
oc edit subscription datagrid -n openshift-operators
```

- Use the Red Hat OpenShift Console.
  1. Navigate to **Operators** > **Installed Operators** > **Data Grid Operator**.
  2. From the **Actions** menu, select **Edit Subscription**.

Labels and annotations with environment variables

```
spec:
  config:
    env:
      - name: INFINISPAN_OPERATOR_TARGET_LABELS
        value: |
         {"service-label1":"value",
         service-label1":"value"}
      - name: INFINISPAN_OPERATOR_POD_TARGET_LABELS
        value: |
```

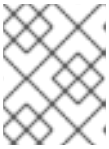
```
    {"pod-label1":"value",
    "pod-label2":"value"}
  - name: INFINISPAN_OPERATOR_TARGET_ANNOTATIONS
    value: |
    {"service-annotation1":"value",
    "service-annotation2":"value"}
  - name: INFINISPAN_OPERATOR_POD_TARGET_ANNOTATIONS
    value: |
    {"pod-annotation1":"value",
    "pod-annotation2":"value"}
```

8.14. ADDING ENVIRONMENT VARIABLES TO DATA GRID PODS

Configure custom environment variables for the Data Grid nodes managed by the Data Grid Operator. You can use literal values to tune behavior, such as **JAVA_OPTS**, or reference values from Secrets and ConfigMaps for sensitive data like credentials. To get more information on valid environment variables accepted by a

Procedure

1. Open your **Infinispan** CR for editing.
2. Define environment variables in the **spec.container.env** section.



NOTE

Adding or modifying environment variables triggers a **rolling update** of the Data Grid cluster to apply the changes to all pods.

Custom environment variables

```
spec:
  container:
    env:
      - name: JAVA_OPTIONS
        value: "-Dkey=value"
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: my-database-secret
            key: password
```

3. Apply your **Infinispan** CR.

Verification

Confirm the environment variables are present in the pods:

```
oc exec <pod-name> -- printenv | grep JAVA_OPTIONS
```



IMPORTANT

Defining **JAVA_OPTIONS** alongside **spec.container.extraJvmOpts** will result in only one of the two being effectively applied. Do not to use them simultaneously. The same restriction applies to **CLI_JAVA_OPTIONS** and **spec.container.cliExtraJvmOpts**.

Additional resources

- JVM settings for Infinispan

8.15. DEFINING ENVIRONMENT VARIABLES IN THE DATA GRID OPERATOR SUBSCRIPTION

You can define environment variables in your Data Grid Operator subscription either when you create or edit the subscription.



NOTE

If you are using the Red Hat OpenShift Console, you must first install the Data Grid Operator and then edit the existing subscription.

spec.config.env field

Includes the **name** and **value** fields to define environment variables.

ADDITIONAL_VARS variable

Includes the names of environment variables in a format of JSON array. Environment variables within the **value** of the **ADDITIONAL_VARS** variable automatically propagate to each Data Grid Server pod managed by the associated Operator.

Prerequisites

- Ensure the Operator Lifecycle Manager (OLM) is installed.
- Have an **oc** client.

Procedure

1. Create a subscription definition YAML for your Data Grid Operator:
   a. Use the **spec.config.env** field to define environment variables.
   b. Within the **ADDITIONAL_VARS** variable, include environment variable names in a JSON array.

subscription-datagrid.yaml

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: datagrid
  namespace: openshift-operators
spec:
  channel: 8.6.x
```

```
  installPlanApproval: Automatic
  name: datagrid
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  config:
    env:
    - name: ADDITIONAL_VARS
      value: "[\"VAR_NAME\", \"ANOTHER_VAR\"]"
    - name: VAR_NAME
      value: $(VAR_NAME_VALUE)
    - name: ANOTHER_VAR
      value: $(ANOTHER_VAR_VALUE)
```

For example, use the environment variables to set the local time zone:

subscription-datagrid.yaml

```
kind: Subscription
spec:
  ...
  config:
    env:
    - name: ADDITIONAL_VARS
      value: "[\"TZ\"]"
    - name: TZ
      value: "JST-9"
```

2. Create a subscription for Data Grid Operator:

```
oc apply -f subscription-datagrid.yaml
```

Verification

- Retrieve the environment variables from the **subscription-datagrid.yaml**:

```
oc get subscription datagrid -n openshift-operators -o jsonpath='{.spec.config.env[*].name}'
```

Next steps

1. Use the **oc edit subscription** command to modify the environment variable:

```
oc edit subscription datagrid -n openshift-operators
```

2. To ensure the changes take effect on your Data Grid clusters, you must recreate the existing clusters. Terminate the pods by deleting the **StatefulSet** associated with the existing **Infinispan** CRs.

- In the Red Hat OpenShift Console, navigate to **Operators** > **Installed Operators** > **Data Grid Operator**. From the **Actions** menu, select **Edit Subscription**.

CHAPTER 9. CONFIGURING AUTHENTICATION

Application users need credentials to access Data Grid clusters. You can use default, generated credentials or add your own.

9.1. DEFAULT CREDENTIALS

Data Grid Operator generates base64-encoded credentials for the following users:

User	Secret name	Description
developer	**infinispan-generated-secret**	Credentials for the default application user.
operator	**infinispan-generated-operator-secret**	Credentials that Data Grid Operator uses to interact with Data Grid resources.

9.2. RETRIEVING CREDENTIALS

Get credentials from authentication secrets to access Data Grid clusters.

Procedure

- Retrieve credentials from authentication secrets.

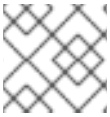
```
oc get secret infinispan-generated-secret
```

Base64-decode credentials.

```
oc get secret infinispan-generated-secret -o jsonpath="{.data.identities\.yaml}" | base64 --decode
```

9.3. ADDING CUSTOM USER CREDENTIALS

Configure access to Data Grid cluster endpoints with custom credentials.



NOTE

Modifying **spec.security.endpointSecretName** triggers a cluster restart.

Procedure

1. Create an **identities.yaml** file with the credentials that you want to add.

```
credentials:
- username: myfirstusername
  password: changeme-one
- username: mysecondusername
  password: changeme-two
```

2. Create an authentication secret from **identities.yaml**.

```
oc create secret generic --from-file=identities.yaml connect-secret
```

3. Specify the authentication secret with **spec.security.endpointSecretName** in your **Infinispan** CR and then apply the changes.

```
spec:
  security:
    endpointSecretName: connect-secret
```

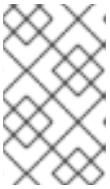
9.4. CHANGING THE OPERATOR PASSWORD

You can change the password for the **operator** user if you do not want to use the automatically generated password.

Procedure

- Update the **password** key in the **infinispan-generated-operator-secret** secret as follows:

```
oc patch secret infinispan-generated-operator-secret -p='{"stringData":{"password":
"supersecretoperatorpassword"}}'
```



NOTE

You should update only the **password** key in the **generated-operator-secret** secret. When you update the password, Data Grid Operator automatically refreshes other keys in that secret.

9.5. DISABLING USER AUTHENTICATION

Allow users to access Data Grid clusters and manipulate data without providing credentials.



IMPORTANT

Do not disable authentication if endpoints are accessible from outside the OpenShift cluster via **spec.expose.type**. You should disable authentication for development environments only.

Procedure

1. Set **false** as the value for the **spec.security.endpointAuthentication** field in your **Infinispan** CR.

```
spec:
  security:
    endpointAuthentication: false
```

2. Apply the changes.

CHAPTER 10. CONFIGURING CLIENT CERTIFICATE AUTHENTICATION

Add client trust stores to your project and configure Data Grid to allow connections only from clients that present valid certificates. This increases security of your deployment by ensuring that clients are trusted by a public certificate authority (CA).

10.1. CLIENT CERTIFICATE AUTHENTICATION

Client certificate authentication restricts in-bound connections based on the certificates that clients present.

You can configure Data Grid to use trust stores with either of the following strategies:

Validate

To validate client certificates, Data Grid requires a trust store that contains any part of the certificate chain for the signing authority, typically the root CA certificate. Any client that presents a certificate signed by the CA can connect to Data Grid.

If you use the **Validate** strategy for verifying client certificates, you must also configure clients to provide valid Data Grid credentials if you enable authentication.

Authenticate

Requires a trust store that contains all public client certificates in addition to the root CA certificate. Only clients that present a signed certificate can connect to Data Grid.

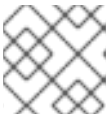
If you use the **Authenticate** strategy for verifying client certificates, you must ensure that certificates contain valid Data Grid credentials as part of the distinguished name (DN).

10.2. ENABLING CLIENT CERTIFICATE AUTHENTICATION

To enable client certificate authentication, you configure Data Grid to use trust stores with either the **Validate** or **Authenticate** strategy.

Procedure

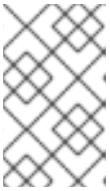
1. Set either **Validate** or **Authenticate** as the value for the **spec.security.endpointEncryption.clientCert** field in your **Infinispan** CR.



NOTE

The default value is **None**.

2. Specify the secret that contains the client trust store with the **spec.security.endpointEncryption.clientCertSecretName** field.
   By default Data Grid Operator expects a trust store secret named **<cluster-name>-client-cert-secret**.



NOTE

The secret must be unique to each **Infinispan** CR instance in the OpenShift cluster. When you delete the **Infinispan** CR, OpenShift also automatically deletes the associated secret.

```
spec:
  security:
    endpointEncryption:
        type: Secret
        certSecretName: tls-secret
        clientCert: Validate
        clientCertSecretName: infinispan-client-cert-secret
```

3. Apply the changes.

Next steps

Provide Data Grid Operator with a trust store that contains all client certificates. Alternatively you can provide certificates in PEM format and let Data Grid generate a client trust store.

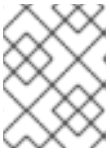
10.3. PROVIDING CLIENT TRUSTSTORES

If you have a trust store that contains the required certificates you can make it available to Data Grid Operator.

Data Grid supports trust stores in **PKCS12** format only.

Procedure

1. Specify the name of the secret that contains the client trust store as the value of the **metadata.name** field.



NOTE

The name must match the value of the **spec.security.endpointEncryption.clientCertSecretName** field.

2. Provide the password for the trust store with the **stringData.truststore-password** field.

3. Specify the trust store with the **data.truststore.p12** field.

```
apiVersion: v1
kind: Secret
metadata:
  name: infinispan-client-cert-secret
type: Opaque
stringData:
    truststore-password: changme
data:
    truststore.p12:  "<base64_encoded_PKCS12_trust_store>"
```

4. Apply the changes.

10.4. PROVIDING CLIENT CERTIFICATES

Data Grid Operator can generate a trust store from certificates in PEM format.

Procedure

1. Specify the name of the secret that contains the client trust store as the value of the **metadata.name** field.



NOTE

The name must match the value of the **spec.security.endpointEncryption.clientCertSecretName** field.

2. Specify the signing certificate, or CA certificate bundle, as the value of the **data.trust.ca** field.
3. If you use the **Authenticate** strategy to verify client identities, add the certificate for each client that can connect to Data Grid endpoints with the **data.trust.cert.<name>** field.



NOTE

Data Grid Operator uses the **<name>** value as the alias for the certificate when it generates the trust store.

4. Optionally provide a password for the trust store with the **stringData.truststore-password** field.
   If you do not provide one, Data Grid Operator sets "password" as the trust store password.

```
apiVersion: v1
kind: Secret
metadata:
  name: infinispan-client-cert-secret
type: Opaque
stringData:
    truststore-password: changme
data:
    trust.ca: "<base64_encoded_CA_certificate>"
    trust.cert.client1: "<base64_encoded_client_certificate>"
    trust.cert.client2: "<base64_encoded_client_certificate>"
```

5. Apply the changes.

CHAPTER 11. CONFIGURING ENCRYPTION

Encrypt connections between clients and Data Grid pods with Red Hat OpenShift service certificates or custom TLS certificates.

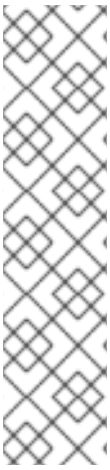
11.1. ENCRYPTION WITH RED HAT OPENSHIFT SERVICE CERTIFICATES

Data Grid Operator automatically generates TLS certificates that are signed by the Red Hat OpenShift service CA. Data Grid Operator then stores the certificates and keys in a secret so you can retrieve them and use with remote clients.

If the Red Hat OpenShift service CA is available, Data Grid Operator adds the following **spec.security.endpointEncryption** configuration to the **Infinispan** CR:

```
spec:
  security:
    endpointEncryption:
      type: Service
      certServiceName: service.beta.openshift.io
      certSecretName: infinispan-cert-secret
```

Field	Description
spec.security.endpointEncryption.certServiceName	Specifies the service that provides TLS certificates.
spec.security.endpointEncryption.certSecretName	Specifies a secret with a service certificate and key in PEM format. Defaults to **<cluster_name>-cert-secret**.



NOTE

Service certificates use the internal DNS name of the Data Grid cluster as the common name (CN), for example:

Subject: CN = example-infinispan.mynamespace.svc

For this reason, service certificates can be fully trusted only inside OpenShift. If you want to encrypt connections with clients running outside OpenShift, you should use custom TLS certificates.

Service certificates are valid for one year and are automatically replaced before they expire.

11.2. RETRIEVING TLS CERTIFICATES

Get TLS certificates from encryption secrets to create client trust stores.

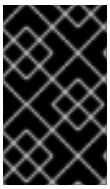
Procedure

- Retrieve **tls.crt** from encryption secrets as follows:

```
oc get secret infinispan-cert-secret -o jsonpath='{.data.tls\.crt}' | base64 --decode > tls.crt
```

11.3. DISABLING ENCRYPTION

You can disable encryption so clients do not need TLS certificates to establish connections with Data Grid.



IMPORTANT

Do not disable encryption if endpoints are accessible from outside the OpenShift cluster via **spec.expose.type**. You should disable encryption for development environments only.

Procedure

1. Set **None** as the value for the **spec.security.endpointEncryption.type** field in your **Infinispan** CR.

```
spec:
  security:
    endpointEncryption:
      type: None
```

2. Apply the changes.

11.4. USING CUSTOM TLS CERTIFICATES

Use custom PKCS12 keystore or TLS certificate/key pairs to encrypt connections between clients and Data Grid clusters.

Prerequisites

- Create either a keystore or certificate secret.



NOTE

The secret must be unique to each **Infinispan** CR instance in the OpenShift cluster. When you delete the **Infinispan** CR, OpenShift also automatically deletes the associated secret.

Procedure

1. Add the encryption secret to your OpenShift namespace, for example:

```
oc apply -f tls_secret.yaml
```

2. Specify the encryption secret with the **spec.security.endpointEncryption.certSecretName** field in your **Infinispan** CR.

```
spec:
  security:
    endpointEncryption:
```

```
    type: Secret
    certSecretName: tls-secret
```

3. Apply the changes.

11.4.1. Custom encryption secrets

Custom encryption secrets that add keystores or certificate/key pairs to secure Data Grid connections must contain specific fields.

Keystore secrets

```
apiVersion: v1
kind: Secret
metadata:
  name: tls-secret
type: Opaque
stringData:
  alias: server
  password: changeme
data:
  keystore.p12:  "MIIKDgIBAzCCCdQGCSqGSIb3DQEHA..."
```

Field	Description
stringData.alias	Specifies an alias for the keystore.
stringData.password	Specifies the keystore password.
data.keystore.p12	Adds a base64-encoded keystore.

Certificate secrets

```
apiVersion: v1
kind: Secret
metadata:
  name: tls-secret
type: Opaque
data:
  tls.key:  "LS0tLS1CRUdJTiBQUk ..."
  tls.crt: "LS0tLS1CRUdJTiBDRVI ..."
```

Field	Description
data.tls.key	Adds a base64-encoded TLS key.
data.tls.crt	Adds a base64-encoded TLS certificate.

CHAPTER 12. CONFIGURING USER ROLES AND PERMISSIONS

Secure access to Data Grid services by configuring role-based access control (RBAC) for users. This requires you to assign roles to users so that they have permission to access caches and Data Grid resources.

12.1. ENABLING SECURITY AUTHORIZATION

By default authorization is disabled to ensure backwards compatibility with **Infinispan** CR instances. Complete the following procedure to enable authorization and use role-based access control (RBAC) for Data Grid users.

Procedure

1. Set **true** as the value for the **spec.security.authorization.enabled** field in your **Infinispan** CR.

```
spec:
  security:
    authorization:
      enabled: true
```

2. Apply the changes.

12.2. USER ROLES AND PERMISSIONS

Data Grid Operator provides a set of default roles that are associated with different permissions.

Table 12.1. Default roles and permissions

Role	Permissions	Description
admin	ALL	Superuser with all permissions including control of the Cache Manager lifecycle.
deployer	ALL_READ, ALL_WRITE, LISTEN, EXEC, MONITOR, CREATE	Can create and delete Data Grid resources in addition to **application** permissions.
application	ALL_READ, ALL_WRITE, LISTEN, EXEC, MONITOR	Has read and write access to Data Grid resources in addition to **observer** permissions. Can also listen to events and execute server tasks and scripts.
observer	ALL_READ, MONITOR	Has read access to Data Grid resources in addition to **monitor** permissions.
monitor	MONITOR	Can view statistics for Data Grid clusters.

Data Grid Operator credentials

Data Grid Operator generates credentials that it uses to authenticate with Data Grid clusters to perform internal operations. By default Data Grid Operator credentials are automatically assigned the **admin** role when you enable security authorization.

Additional resources

- How security authorization works (*Data Grid Security Guide*).

12.3. ASSIGNING ROLES AND PERMISSIONS TO USERS

Assign users with roles that control whether users are authorized to access Data Grid cluster resources. Roles can have different permission levels, from read-only to unrestricted access.



NOTE

Users gain authorization implicitly. For example, "admin" gets **admin** permissions automatically. A user named "deployer" has the **deployer** role automatically, and so on.

Procedure

1. Create an **identities.yaml** file that assigns roles to users.

```
credentials:
  - username: admin
    password: changeme
  - username: my-user-1
    password: changeme
    roles:
      - admin
  - username: my-user-2
    password: changeme
    roles:
      - monitor
```

2. Create an authentication secret from **identities.yaml**.
   If necessary, delete the existing secret first.

```
oc delete secret connect-secret --ignore-not-found
oc create secret generic --from-file=identities.yaml connect-secret
```

3. Specify the authentication secret with **spec.security.endpointSecretName** in your **Infinispan** CR and then apply the changes.

```
spec:
  security:
    endpointSecretName: connect-secret
```

12.4. ADDING CUSTOM ROLES AND PERMISSIONS

You can define custom roles with different combinations of permissions.

Procedure

1. Open your **Infinispan** CR for editing.
2. Specify custom roles and their associated permissions with the **spec.security.authorization.roles** field.

```
spec:
  security:
    authorization:
      enabled: true
      roles:
        - name: my-role-1
          permissions:
            - ALL
        - name: my-role-2
          permissions:
            - READ
            - WRITE
```

3. Apply the changes.

CHAPTER 13. CONFIGURING NETWORK ACCESS TO DATA GRID

Expose Data Grid clusters so you can access Data Grid Console, the Data Grid command line interface (CLI), REST API, and Hot Rod endpoint.

13.1. GETTING THE SERVICE FOR INTERNAL CONNECTIONS

By default, Data Grid Operator creates a service that provides access to Data Grid clusters from clients running on OpenShift.

This internal service has the same name as your Data Grid cluster, for example:

```
metadata:
  name: infinispan
```

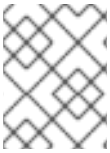
Procedure

- Check that the internal service is available as follows:

```
oc get services
```

13.2. EXPOSING DATA GRID THROUGH A LOADBALANCER SERVICE

Use a **LoadBalancer** service to make Data Grid clusters available to clients running outside OpenShift.



NOTE

To access Data Grid with unencrypted Hot Rod client connections you must use a **LoadBalancer** service.

Procedure

1. Include **spec.expose** in your **Infinispan** CR.
2. Specify **LoadBalancer** as the service type with the **spec.expose.type** field.
3. Optionally specify the network port where the service is exposed with the **spec.expose.port** field.

```
spec:
  expose:
    type: LoadBalancer
    port: 65535
```

4. Apply the changes.
5. Verify that the **-external** service is available.

```
oc get services | grep external
```

13.3. EXPOSING DATA GRID THROUGH A NODEPORT SERVICE

Use a **NodePort** service to expose Data Grid clusters on the network.

Procedure

1. Include **spec.expose** in your **Infinispan** CR.
2. Specify **NodePort** as the service type with the **spec.expose.type** field.
3. Configure the port where Data Grid is exposed with the **spec.expose.nodePort** field.

```
spec:
  expose:
    type: NodePort
    nodePort: 30000
```

4. Apply the changes.
5. Verify that the **-external** service is available.

```
oc get services | grep external
```

13.4. EXPOSING DATA GRID THROUGH A ROUTE

Use an OpenShift **Route** with passthrough encryption to make Data Grid clusters available on the network.



NOTE

To access Data Grid with Hot Rod client, you must configure TLS with SNI.

Procedure

1. Include **spec.expose** in your **Infinispan** CR.
2. Specify **Route** as the service type with the **spec.expose.type** field.
3. Optionally add a hostname with the **spec.expose.host** field.

```
spec:
  expose:
    type: Route
    host: www.example.org
```

4. Apply the changes.
5. Verify that the route is available.

```
oc get routes
```

Route ports

When you create a **Route**, it exposes a port on the network that accepts client connections and redirects traffic to Data Grid services that listen on port **11222**.

The port where the **Route** is available depends on whether you use encryption or not.

Port	Description
80	Encryption is disabled.
443	Encryption is enabled.

13.5. NETWORK SERVICES

Reference information for network services that Data Grid Operator creates and manages.

Service	Port	Protocol	Description
<cluster_name>	**11222**	TCP	Access to Data Grid endpoints within the OpenShift cluster or from an OpenShift **Route**.
<cluster_name>-admin	**11223**	TCP	Access to Data Grid endpoints within the OpenShift cluster for internal Data Grid Operator use. This port utilises a different security-realm to port 11222 and should not be accessed by user applications.
<cluster_name>-ping	**8888**	TCP	Cluster discovery for Data Grid pods.
<cluster_name>-external	**11222**	TCP	Access to Data Grid endpoints from a **LoadBalancer** or **NodePort** service.
<cluster_name>-site	**7900**	TCP	JGroups RELAY2 channel for cross-site communication.



NOTE

The Data Grid Console should only be accessed via OpenShift services or an OpenShift **Route** exposing port 11222.

CHAPTER 14. SETTING UP CROSS-SITE REPLICATION

Ensure availability with Data Grid Operator by configuring geographically distributed clusters as a unified service.

You can configure clusters to perform cross-site replication with:

- Connections that Data Grid Operator manages.
- Connections that you configure and manage.



NOTE

You can use both managed and manual connections for Data Grid clusters in the same **Infinispan** CR. You must ensure that Data Grid clusters establish connections in the same way at each site.

14.1. CROSS-SITE REPLICATION EXPOSE TYPES

You can use a **NodePort** service, a **LoadBalancer** service, or an OpenShift **Route** to handle network traffic for backup operations between Data Grid clusters. Before you start setting up cross-site replication you should determine what expose type is available for your Red Hat OpenShift cluster. In some cases you may require an administrator to provision services before you can configure an expose type.

NodePort

A **NodePort** is a service that accepts network traffic at a static port, in the **30000** to **32767** range, on an IP address that is available externally to the OpenShift cluster.

To use a **NodePort** as the expose type for cross-site replication, an administrator must provision external IP addresses for each OpenShift node. In most cases, an administrator must also configure DNS routing for those external IP addresses.

LoadBalancer

A **LoadBalancer** is a service that directs network traffic to the correct node in the OpenShift cluster.

Whether you can use a **LoadBalancer** as the expose type for cross-site replication depends on the host platform. AWS supports network load balancers (NLB) while some other cloud platforms do not. To use a **LoadBalancer** service, an administrator must first create an ingress controller backed by an NLB.

Route

An OpenShift **Route** allows Data Grid clusters to connect with each other through a public secure URL.

Data Grid uses TLS with the SNI header to send backup requests between clusters through an OpenShift **Route**. To do this you must add a keystore with TLS certificates so that Data Grid can encrypt network traffic for cross-site replication.

When you specify **Route** as the expose type for cross-site replication, Data Grid Operator creates a route with TLS passthrough encryption for each Data Grid cluster that it manages. You can specify a hostname for the **Route** but you cannot specify a **Route** that you have already created.

Additional resources

- Configuring ingress cluster traffic overview

14.2. MANAGED CROSS-SITE REPLICATION

Data Grid Operator can discover Data Grid clusters running in different data centers to form global clusters.

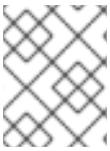
When you configure managed cross-site connections, Data Grid Operator creates router pods in each Data Grid cluster. Data Grid pods use the **<cluster_name>-site** service to connect to these router pods and send backup requests.

Router pods maintain a record of all pod IP addresses and parse RELAY message headers to forward backup requests to the correct Data Grid cluster. If a router pod crashes then all Data Grid pods start using any other available router pod until OpenShift restores it.



IMPORTANT

To manage cross-site connections, Data Grid Operator uses the Kubernetes API. Each OpenShift cluster must have network access to the remote Kubernetes API and a service account token for each backup cluster.



NOTE

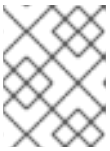
Data Grid clusters do not start running until Data Grid Operator discovers all backup locations that you configure.

14.2.1. Creating service account tokens for managed cross-site connections

Generate service account tokens on OpenShift clusters that allow Data Grid Operator to automatically discover Data Grid clusters and manage cross-site connections.

Prerequisites

- Ensure all OpenShift clusters have access to the Kubernetes API.
  Data Grid Operator uses this API to manage cross-site connections.



NOTE

Data Grid Operator does not modify remote Data Grid clusters. The service account tokens provide read-only access through the Kubernetes API.

Procedure

1. Log in to an OpenShift cluster.
2. Create a service account.
   For example, create a service account at **LON**:

```
oc create sa -n <namespace> lon
```

3. Add the view role to the service account with the following command:

```
oc policy add-role-to-user view -n <namespace> -z lon
```

4. If you use a **NodePort** service to expose Data Grid clusters on the network, you must also add the **cluster-reader** role to the service account:

```
oc adm policy add-cluster-role-to-user cluster-reader -z lon -n <namespace>
```

5. Repeat the preceding steps on your other OpenShift clusters.
6. Exchange service account tokens on each OpenShift cluster.

14.2.2. Exchanging service account tokens

Generate service account tokens on your OpenShift clusters and add them into secrets at each backup location. The tokens that you generate in this procedure do not expire. For bound service account tokens, see Exchanging bound service account tokens .

Prerequisites

- You have created a service account.

Procedure

1. Log in to your OpenShift cluster.
2. Create a service account token secret file as follows:

sa-token.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: ispn-xsite-sa-token (1)
  annotations:
    kubernetes.io/service-account.name: "<service-account>" (2)
type: kubernetes.io/service-account-token
```

(1) Specifies the name of the secret.

(2) Specifies the service account name.

3. Create the secret in your OpenShift cluster:

```
oc -n <namespace> create -f sa-token.yaml
```

4. Retrieve the service account token:

```
oc -n <namespace> get secrets ispn-xsite-sa-token -o jsonpath="{.data.token}" | base64 -d
```

The command prints the token in the terminal.

5. Copy the token for deployment in the backup OpenShift cluster.
6. Log in to the backup OpenShift cluster.

7. Add the service account token for a backup location:

```
oc -n <namespace> create secret generic <token-secret> --from-literal=token=<token>
```

The **<token-secret>** is the name of the secret configured in the **Infinispan** CR.

Next steps

- Repeat the preceding steps on your other OpenShift clusters.

Additional resources

- Creating a service account token secret

14.2.3. Exchanging bound service account tokens

Create service account tokens with a limited lifespan and add them into secrets at each backup location. You must refresh the token periodically to prevent Data Grid Operator from losing access to the remote OpenShift cluster. For non-expiring tokens, see Exchanging service account tokens .

Prerequisites

- You have created a service account.

Procedure

1. Log in to your OpenShift cluster.
2. Create a bound token for the service account:

```
oc -n <namespace> create token <service-account>
```



NOTE

By default, service account tokens are valid for one hour. Use the command option **--duration** to specify the lifespan in seconds..

The command prints the token in the terminal.

3. Copy the token for deployment in the backup OpenShift cluster(s).
4. Log in to the backup OpenShift cluster.
5. Add the service account token for a backup location:

```
oc -n <namespace> create secret generic <token-secret> --from-literal=token=<token>
```

The **<token-secret>** is the name of the secret configured in the **Infinispan** CR.

6. Repeat the steps on other OpenShift clusters.

Deleting expired tokens

When a token expires, delete the expired token secret, and then repeat the procedure to generate and exchange a new one.

1. Log in to the backup OpenShift cluster.
2. Delete the expired secret **<token-secret>**:

```
oc -n <namespace> delete secrets <token-secret>
```

3. Repeat the procedure to create a new token and generate a new **<token-secret>**.

Additional resources

- Creating bound service account tokens

14.2.4. Configuring managed cross-site connections

Configure Data Grid Operator to establish cross-site views with Data Grid clusters.

Prerequisites

- Determine a suitable expose type for cross-site replication.
  If you use an OpenShift **Route** you must add a keystore with TLS certificates and secure cross-site connections.
- Create and exchange Red Hat OpenShift service account tokens for each Data Grid cluster.

Procedure

1. Create an **Infinispan** CR for each Data Grid cluster.
2. Specify the name of the local site with **spec.service.sites.local.name**.
3. Configure the expose type for cross-site replication.
   a. Set the value of the **spec.service.sites.local.expose.type** field to one of the following:
      - **NodePort**
      - **LoadBalancer**
      - **Route**
   b. Optionally specify a port or custom hostname with the following fields:
      - **spec.service.sites.local.expose.nodePort** if you use a **NodePort** service.
      - **spec.service.sites.local.expose.port** if you use a **LoadBalancer** service.
      - **spec.service.sites.local.expose.routeHostName** if you use an OpenShift **Route**.
4. Specify the number of pods that can send RELAY messages with the **service.sites.local.maxRelayNodes** field.

TIP

Configure all pods in your cluster to send **RELAY** messages for better performance. If all pods send backup requests directly, then no pods need to forward backup requests.

5. Provide the name, URL, and secret for each Data Grid cluster that acts as a backup location with **spec.service.sites.locations**.

6. If Data Grid cluster names or namespaces at the remote site do not match the local site, specify those values with the **clusterName** and **namespace** fields.
   The following are example **Infinispan** CR definitions for **LON** and **NYC**:

   - LON

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 3
  version: <Data Grid_version>
  service:
    type: DataGrid
    sites:
      local:
        name: LON
        expose:
          type: LoadBalancer
          port: 65535
        maxRelayNodes: 1
      locations:
        - name: NYC
          clusterName: <nyc_cluster_name>
          namespace: <nyc_cluster_namespace>
          url: openshift://api.rhdg-nyc.openshift-aws.myhost.com:6443
          secretName: nyc-token
  logging:
    categories:
      org.jgroups.protocols.TCP: error
      org.jgroups.protocols.relay.RELAY2: error
```

   - NYC

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: nyc-cluster
spec:
  replicas: 2
  version: <Data Grid_version>
  service:
    type: DataGrid
    sites:
      local:
        name: NYC
```

```
      expose:
        type: LoadBalancer
        port: 65535
      maxRelayNodes: 1
    locations:
      - name: LON
        clusterName: infinispan
        namespace: rhdg-namespace
        url: openshift://api.rhdg-lon.openshift-aws.myhost.com:6443
        secretName: lon-token
logging:
  categories:
    org.jgroups.protocols.TCP: error
    org.jgroups.protocols.relay.RELAY2: error
```



IMPORTANT

Be sure to adjust logging categories in your **Infinispan** CR to decrease log levels for JGroups TCP and RELAY2 protocols. This prevents a large number of log files from uses container storage.

```
spec:
  logging:
    categories:
      org.jgroups.protocols.TCP: error
      org.jgroups.protocols.relay.RELAY2: error
```

7. Configure your **Infinispan** CRs with any other Data Grid service resources and then apply the changes.
8. Verify that Data Grid clusters form a cross-site view.
   a. Retrieve the **Infinispan** CR.

   ```
   oc get infinispan -o yaml
   ```

   b. Check for the **type: CrossSiteViewFormed** condition.

Next steps

If your clusters have formed a cross-site view, you can start adding backup locations to caches.

Additional resources

- Data Grid guide to cross-site replication

14.3. MANUALLY CONFIGURING CROSS-SITE CONNECTIONS

You can specify static network connection details to perform cross-site replication with Data Grid clusters running outside OpenShift. Manual cross-site connections are necessary in any scenario where access to the Kubernetes API is not available outside the OpenShift cluster where Data Grid runs.

Prerequisites

- Determine a suitable expose type for cross-site replication.
  If you use an OpenShift **Route** you must add a keystore with TLS certificates and secure cross-site connections.

- Ensure you have the correct host names and ports for each Data Grid cluster and each **<cluster-name>-site** service.
  Manually connecting Data Grid clusters to form cross-site views requires predictable network locations for Data Grid services, which means you need to know the network locations before they are created.

Procedure

1. Create an **Infinispan** CR for each Data Grid cluster.
2. Specify the name of the local site with **spec.service.sites.local.name**.
3. Configure the expose type for cross-site replication.
   a. Set the value of the **spec.service.sites.local.expose.type** field to one of the following:
      - **NodePort**
      - **LoadBalancer**
      - **Route**
   b. Optionally specify a port or custom hostname with the following fields:
      - **spec.service.sites.local.expose.nodePort** if you use a **NodePort** service.
      - **spec.service.sites.local.expose.port** if you use a **LoadBalancer** service.
      - **spec.service.sites.local.expose.routeHostName** if you use an OpenShift **Route**.
4. Provide the name and static URL for each Data Grid cluster that acts as a backup location with **spec.service.sites.locations**, for example:

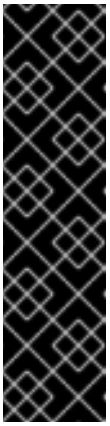
   - LON

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 3
  version: <Data Grid_version>
  service:
    type: DataGrid
    sites:
      local:
        name: LON
        expose:
          type: LoadBalancer
          port: 65535
        maxRelayNodes: 1
      locations:
        - name: NYC
```

```
        url: infinispan+xsite://infinispan-nyc.myhost.com:7900
  logging:
    categories:
      org.jgroups.protocols.TCP: error
      org.jgroups.protocols.relay.RELAY2: error
```

- NYC

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  version: <Data Grid_version>
  service:
    type: DataGrid
    sites:
      local:
        name: NYC
        expose:
          type: LoadBalancer
          port: 65535
        maxRelayNodes: 1
      locations:
        - name: LON
          url: infinispan+xsite://infinispan-lon.myhost.com
  logging:
    categories:
      org.jgroups.protocols.TCP: error
      org.jgroups.protocols.relay.RELAY2: error
```



IMPORTANT

Be sure to adjust logging categories in your **Infinispan** CR to decrease log levels for JGroups TCP and RELAY2 protocols. This prevents a large number of log files from uses container storage.

```
spec:
  logging:
    categories:
      org.jgroups.protocols.TCP: error
      org.jgroups.protocols.relay.RELAY2: error
```

5. Configure your **Infinispan** CRs with any other Data Grid service resources and then apply the changes.

6. Verify that Data Grid clusters form a cross-site view.

   a. Retrieve the **Infinispan** CR.

   ```
   oc get infinispan -o yaml
   ```

   b. Check for the **type: CrossSiteViewFormed** condition.

Next steps

If your clusters have formed a cross-site view, you can start adding backup locations to caches.

Additional resources

- Data Grid guide to cross-site replication

14.4. ALLOCATING CPU AND MEMORY FOR GOSSIP ROUTER POD

Allocate CPU and memory resources to Data Grid Gossip router.

Prerequisite

- Have Gossip router enabled. The **service.sites.local.discovery.launchGossipRouter** property must be set to **true**, which is the default value.

Procedure

1. Allocate the number of CPU units using the **service.sites.local.discovery.cpu** field.
2. Allocate the amount of memory, in bytes, using the **service.sites.local.discovery.memory** field.
   The **cpu** and **memory** fields have values in the format of **<limit>:<requests>**. For example, **cpu: "2000m:1000m"** limits pods to a maximum of **2000m** of CPU and requests **1000m** of CPU for each pod at startup. Specifying a single value sets both the limit and request.
3. Apply your **Infinispan** CR.

```
spec:
  service:
    type: DataGrid
    sites:
      local:
        name: LON
        discovery:
          launchGossipRouter: true
          memory: "2Gi:1Gi"
          cpu: "2000m:1000m"
```

14.5. DISABLING LOCAL GOSSIP ROUTER AND SERVICE

The Data Grid Operator starts a Gossip router on each site, but you only need a single Gossip router to manage traffic between the Data Grid cluster members. You can disable the additional Gossip routers to save resources.

For example, you have Data Grid clusters in **LON** and **NYC** sites. The following procedure shows how you can disable Gossip router in **LON** site and connect to **NYC** that has the Gossip router enabled.

Procedure

1. Create an **Infinispan** CR for each Data Grid cluster.

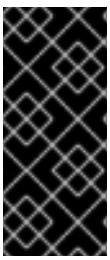
2. Specify the name of the local site with the **spec.service.sites.local.name** field.
3. For the **LON** cluster, set **false** as the value for the **spec.service.sites.local.discovery.launchGossipRouter** field.
4. For the **LON** cluster, specify the **url** with the **spec.service.sites.locations.url** to connect to the **NYC**.
5. In the **NYC** configuration, do not specify the **spec.service.sites.locations.url**.

LON

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 3
  service:
    type: DataGrid
    sites:
      local:
        name: LON
        discovery:
          launchGossipRouter: false
      locations:
        - name: NYC
          url: infinispan+xsite://infinispan-nyc.myhost.com:7900
```

NYC

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 3
  service:
    type: DataGrid
    sites:
      local:
        name: NYC
      locations:
        - name: LON
```



IMPORTANT

If you have three or more sites, Data Grid recommends to keep the Gossip router enabled on all the remote sites. When you have multiple Gossip routers and one of them becomes unavailable, the remaining routers continue exchanging messages. If a single Gossip router is defined, and it becomes unavailable, the connection between the remote sites breaks.

Next steps

If your clusters have formed a cross-site view, you can start adding backup locations to caches.

Additional resources

- Data Grid cross-site replication

14.6. RESOURCES FOR CONFIGURING CROSS-SITE REPLICATION

The following tables provides fields and descriptions for cross-site resources.

Table 14.1. service.type

Field	Description
service.type: DataGrid	Data Grid supports cross-site replication with Data Grid service clusters only.

Table 14.2. service.sites.local

Field	Description
service.sites.local.name	Names the local site where a Data Grid cluster runs.
service.sites.local.maxRelayNodes	Specifies the maximum number of pods that can send RELAY messages for cross-site replication. The default value is **1**.
service.sites.local.discovery.launchGossipRouter	If **false**, the cross-site services and the Gossip router pod are not created in the local site. The default value is **true**.
service.sites.local.discovery.memory	Allocates the amount of memory in bytes. It uses the following format **<limit>:<requests>** (example **"2Gi:1Gi"**).
service.sites.local.discovery.cpu	Allocates the number of CPU units. It uses the following format **<limit>:<requests>** (example **"2000m:1000m"**).
service.sites.local.expose.type	Specifies the network service for cross-site replication. Data Grid clusters use this service to communicate and perform backup operations. You can set the value to **NodePort**, **LoadBalancer**, or **Route**.
service.sites.local.expose.nodePort	Specifies a static port within the default range of **30000** to **32767** if you expose Data Grid through a **NodePort** service. If you do not specify a port, the platform selects an available one.

Field	Description
service.sites.local.expose.port	Specifies the network port for the service if you expose Data Grid through a **LoadBalancer** service. The default port is **7900**.
service.sites.local.expose.routeHostName	Specifies a custom hostname if you expose Data Grid through an OpenShift **Route**. If you do not set a value then OpenShift generates a hostname.

Table 14.3. service.sites.locations

Field	Description
service.sites.locations	Provides connection information for all backup locations.
service.sites.locations.name	Specifies a backup location that matches **.spec.service.sites.local.name**.
service.sites.locations.url	Specifies the URL of the Kubernetes API for managed connections or a static URL for manual connections. Use **openshift://** to specify the URL of the Kubernetes API for an OpenShift cluster. Note that the **openshift://** URL must present a valid, CA-signed certificate. You cannot use self-signed certificates. Use the **infinispan+xsite://<hostname>:<port>** format for static hostnames and ports. The default port is **7900**.
service.sites.locations.secretName	Specifies the secret that contains the service account token for the backup site.
service.sites.locations.clusterName	Specifies the cluster name at the backup location if it is different to the cluster name at the local site.
service.sites.locations.namespace	Specifies the namespace of the Data Grid cluster at the backup location if it does not match the namespace at the local site.

Managed cross-site connections

```
spec:
  service:
    type: DataGrid
```

```
  sites:
    local:
      name: LON
      expose:
        type: LoadBalancer
      maxRelayNodes: 1
    locations:
    - name: NYC
      clusterName: <nyc_cluster_name>
      namespace: <nyc_cluster_namespace>
      url: openshift://api.site-b.devcluster.openshift.com:6443
      secretName: nyc-token
```

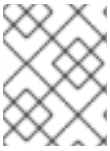
Manual cross-site connections

```
spec:
  service:
    type: DataGrid
    sites:
      local:
        name: LON
        expose:
          type: LoadBalancer
          port: 65535
        maxRelayNodes: 1
      locations:
      - name: NYC
        url: infinispan+xsite://infinispan-nyc.myhost.com:7900
```

14.7. SECURING CROSS-SITE CONNECTIONS

Add keystores and trust stores so that Data Grid clusters can secure cross-site replication traffic.

You must add a keystore to use an OpenShift **Route** as the expose type for cross-site replication. Securing cross-site connections is optional if you use a **NodePort** or **LoadBalancer** as the expose type.



NOTE

Cross-site replication does not support the OpenShift CA service. You must provide your own certificates.

Procedure

1. Create the key store and trust store using Keytool (certificate management utility included with Java).
   a. Generate a self-signed certificate.

```
keytool -genkey -alias server -keyalg RSA -storetype PKCS12 -keystore ./keystore.p12 -validity <days> -storepass <password>
```

   b. Export the certificate from the key store, so that it can be present in the trust store

```
keytool -export -alias server -keystore ./keystore.p12 -storetype PKCS12 -rfc -file
~/server_cert.pem -storepass <password>
```

c. Create the trust store and import the certificate

```
keytool -importcert -alias server -keystore ./truststore.p12 -storetype PKCS12 -file
~/server_cert.pem -storepass <password> -noprompt
```

2. Create cross-site encryption secret for each Data Grid cluster.

a. Create keystore secrets.

```
oc create secret generic keystore-tls-secret \
--from-file=keystore.p12=./keystore.p12 \
--from-literal=password=<password> \
--from-literal=type=pkcs12
```

b. Create trust store secrets.

```
oc create secret generic truststore-tls-secret \
--from-file=truststore.p12=./truststore.p12 \
--from-literal=password=<password> \
--from-literal=type=pkcs12
```

3. Modify the Infinispan CR for each Data Grid cluster to specify the secret name for the **encryption.transportKeyStore.secretName** and **encryption.routerKeyStore.secretName** fields.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  version: <Data Grid_version>
  expose:
    type: LoadBalancer
  service:
    type: DataGrid
    sites:
      local:
        name: SiteA
        # ...
        encryption:
          protocol: TLSv1.3
          transportKeyStore:
            secretName: keystore-tls-secret
            alias: server
            filename: keystore.p12
          routerKeyStore:
            secretName: keystore-tls-secret
            alias: server
            filename: keystore.p12
          trustStore:
```

```
      secretName: truststore-tls-secret
      filename: truststore.p12
locations:
  # ...
```



NOTE

The procedure describes the easier way to encrypt the communication. More complex setups are possible in the CR, for example, by setting different certificates for Data Grid Pod and Gossip Router Pod. Ensure the truststore contains all the required certificate chain for mutual TLS authentication.

14.7.1. Resources for configuring cross-site encryption

The following tables provides fields and descriptions for encrypting cross-site connections.

Table 14.4. service.type.sites.local.encryption

Field	Description
service.type.sites.local.encryption.protocol	Specifies the TLS protocol to use for cross-site connections. The default value is **TLSv1.3** but you can set **TLSv1.2** if required.
service.type.sites.local.encryption.transportKeyStore	Configures a keystore secret for relay pods.
service.type.sites.local.encryption.routerKeyStore	Configures a keystore secret for router pods.
service.type.sites.local.encryption.trustStore	Configures a trust store secret for relay pods and router pods.

Table 14.5. service.type.sites.local.encryption.transportKeyStore

Field	Description
secretName	Specifies the secret that contains a keystore that relay pods can use to encrypt and decrypt RELAY messages. This field is required.
alias	Optionally specifies the alias of the certificate in the keystore. The default value is **transport**.
filename	Optionally specifies the filename of the keystore. The default value is **keystore.p12**.

Table 14.6. service.type.sites.local.encryption.routerKeyStore

Field	Description
secretName	Specifies the secret that contains a keystore that router pods can use to encrypt and decrypt RELAY messages. This field is required.
alias	Optionally specifies the alias of the certificate in the keystore. The default value is **router**.
filename	Optionally specifies the filename of the keystore. The default value is **keystore.p12**.

Table 14.7. service.type.sites.local.encryption.trustStore

Field	Description
secretName	Specifies the secret that contains a trust store to verify public certificates for relay pods and router pods. This field is required.
filename	Optionally specifies the filename of the trust store. The default value is **truststore.p12**.

14.7.2. Cross-site encryption secrets

Cross-site replication encryption secrets add keystores and trust store for securing cross-site connections.

Cross-site encryption secrets

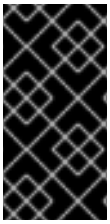
```
apiVersion: v1
kind: Secret
metadata:
  name: tls-secret
type: Opaque
stringData:
  password: changeme
  type: pkcs12
data:
  <file-name>: "MIIKDgIBAzCCCdQGCSqGSIb3DQEHA..."
```

Field	Description
stringData.password	Specifies the password for the keystore or trust store.
stringData.type	Optionally specifies the keystore or trust store type. The default value is **pkcs12**.

Field	Description
data.<file-name>	Adds a base64-encoded keystore or trust store.

14.8. CONFIGURING SITES IN THE SAME OPENSHIFT CLUSTER

For evaluation and demonstration purposes, you can configure Data Grid to back up between pods in the same OpenShift cluster.



IMPORTANT

Using **ClusterIP** as the expose type for cross-site replication is intended for demonstration purposes only. It would be appropriate to use this expose type only to perform a temporary proof-of-concept deployment on a laptop or something of that nature.

Procedure

1. Create an **Infinispan** CR for each Data Grid cluster.
2. Specify the name of the local site with **spec.service.sites.local.name**.
3. Set **ClusterIP** as the value of the **spec.service.sites.local.expose.type** field.
4. Provide the name of the Data Grid cluster that acts as a backup location with **spec.service.sites.locations.clusterName**.
5. If both Data Grid clusters have the same name, specify the namespace of the backup location with **spec.service.sites.locations.namespace**.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: example-clustera
spec:
  replicas: 1
  expose:
    type: LoadBalancer
  service:
    type: DataGrid
    sites:
      local:
        name: SiteA
        expose:
          type: ClusterIP
        maxRelayNodes: 1
      locations:
        - name: SiteB
          clusterName: example-clusterb
          namespace: cluster-namespace
```

6. Configure your **Infinispan** CRs with any other Data Grid service resources and then apply the changes.
7. Verify that Data Grid clusters form a cross-site view.
   a. Retrieve the **Infinispan** CR.

```
oc get infinispan -o yaml
```

   b. Check for the **type: CrossSiteViewFormed** condition.

CHAPTER 15. MONITORING DATA GRID SERVICES

Data Grid exposes metrics that can be used by Prometheus for monitoring and visualizing the cluster state.



NOTE

This documentation explains how to set up monitoring on OpenShift Container Platform. If you're working with community Prometheus deployments, you might find these instructions useful as a general guide. However you should refer to the Prometheus documentation for installation and usage instructions.

See the Prometheus Operator documentation.

15.1. CREATING A PROMETHEUS SERVICE MONITOR

Data Grid Operator automatically creates a Prometheus **ServiceMonitor** that scrapes metrics from your Data Grid cluster.

Procedure

Enable monitoring for user-defined projects on OpenShift Container Platform.

When the Operator detects an **Infinispan** CR with the monitoring annotation set to **true**, which is the default, Data Grid Operator does the following:

- Creates a **ServiceMonitor** named **<cluster_name>-monitor**.
- Adds the **infinispan.org/monitoring: 'true'** annotation to your **Infinispan** CR metadata, if the value is not already explicitly set:

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
  annotations:
    infinispan.org/monitoring: 'true'
```



NOTE

To authenticate with Data Grid, Prometheus uses the **operator** credentials.

Verification

You can check that Prometheus is scraping Data Grid metrics as follows:

1. In the OpenShift Web Console, select the **</> Developer** perspective and then select **Monitoring**.
2. Open the **Dashboard** tab for the namespace where your Data Grid cluster runs.
3. Open the **Metrics** tab and confirm that you can query Data Grid metrics such as:

```
vendor_cache_manager_default_cluster_size
```

Additional resources

- Enabling monitoring for user-defined projects

15.1.1. Disabling the Prometheus service monitor

You can disable the **ServiceMonitor** if you do not want Prometheus to scrape metrics for your Data Grid cluster.

Procedure

1. Set **'false'** as the value for the **infinispan.org/monitoring** annotation in your **Infinispan** CR.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
  annotations:
    infinispan.org/monitoring: 'false'
```

2. Apply the changes.

15.1.2. Configuring Service Monitor Target Labels

You can configure the generated **ServiceMonitor** to propagate Service labels to the underlying metrics using the ServiceMonitor **spec.targetLabels** field. Use the Service labels to filter and aggregate the metrics collected from the monitored endpoints.

Procedure

1. Define labels to apply to your service by setting the **infinispan.org/targetLabels** annotation in your **Infinispan** CR.

2. Specify a comma-separated list of the labels required in your metrics using the **infinispan.org/serviceMonitorTargetLabels** annotation on your **Infinispan** CR.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
  annotations:
    infinispan.org/targetLabels: "label1,label2,label3"
    infinispan.org/serviceMonitorTargetLabels: "label1,label2"
```

3. Apply the changes.

15.2. ENABLING JMX REMOTE PORTS FOR DATA GRID CLUSTERS

Enable JMX remote ports to expose Data Grid MBeans and to integrate Data Grid with external monitoring systems such as Cryostat.

When you enable JMX for Data Grid cluster, the following occurs:

1. Each Data Grid server pod exposes an authenticated JMX endpoint on port **9999** utilizing the "admin" security-realm, which includes the Operator user credentials.
2. The **<cluster-name>-admin** Service exposes port **9999**.



NOTE

You can enable or disable JMX only during the creation of the **Infinispan** CR. Once the CR instance is created, you cannot modify the JMX settings.

Procedure

1. Enable JMX in your **Infinispan** CR.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  jmx:
    enabled: true
```

2. Retrieve the Operator user credentials to authenticate client JMX connections.

```
oc get secret infinispan-generated-operator-secret -o jsonpath="{.data.identities\.yaml}" |
base64 --decode
```

Additional resources

- Enabling JMX statistics

15.3. SETTING UP JFR RECORDINGS WITH CRYOSTAT

Enable JDK Flight Recorder (JFR) monitoring for your Data Grid clusters that run on OpenShift.

JFR recordings with Cryostat

JFR provides insights into various aspects of JVM performance to ease cluster inspection and debugging. Depending on your requirements, you can store and analyze your recordings using the integrated tools provided by Cryostat or export the recordings to an external monitoring application.

Prerequisites

- Install the Cryostat Operator. You can install the Cryostat Operator in your OpenShift project by using Operator Lifecycle Manager (OLM).
- Have JMX enabled on your Data Grid cluster. You must enable JMX before deploying the cluster, as JMX settings cannot be modified after deployment.

Procedure

1. Create a Cryostat CR in the same namespace as your **Infinispan** CR.

```
apiVersion: operator.cryostat.io/v1beta1
```

```
kind: Cryostat
metadata:
  name: cryostat-sample
spec:
  minimal: false
  enableCertManager: true
```



NOTE

The Cryostat Operator requires cert-manager for traffic encryption. If the cert-manager is enabled but not installed, the deployment fails. For details, see the Installing Cryostat guide.

2. Wait for the **Cryostat** CR to be ready.

```
oc wait -n <namespace> --for=condition=MainDeploymentAvailable cryostat/cryostat-sample
```

3. Open the Cryostat **status.applicationUrl**.

```
oc -n <namespace> get cryostat cryostat-sample
```

4. Retrieve the Operator user credentials to authenticate client JMX connections in the Cryostat UI.

```
oc get secret infinispan-generated-operator-secret -o jsonpath="{.data.identities\.yaml}" | base64 --decode
```

5. In the Cryostat UI, navigate to the **Security** menu.
6. In the **Store Credentials** window, click the **Add** button. The **Store Credentials** window opens.
7. In the **Match Expression** filed, enter match expression details in the following format:

```
target.labels['infinispan_cr'] == '<cluster_name>'
```

Additional resources

- Installing Cryostat
- Configuring Cryostat Credentials
- Enabling JMX remote ports for Data Grid clusters

CHAPTER 16. GUARANTEEING AVAILABILITY WITH ANTI-AFFINITY

Kubernetes includes anti-affinity capabilities that protect workloads from single points of failure.

16.1. ANTI-AFFINITY STRATEGIES

Each Data Grid node in a cluster runs in a pod that runs on an OpenShift node in a cluster. Each Red Hat OpenShift node runs on a physical host system. Anti-affinity works by distributing Data Grid nodes across OpenShift nodes, ensuring that your Data Grid clusters remain available even if hardware failures occur.

Data Grid Operator offers two anti-affinity strategies:

kubernetes.io/hostname

Data Grid replica pods are scheduled on different OpenShift nodes.

topology.kubernetes.io/zone

Data Grid replica pods are scheduled across multiple zones.

Fault tolerance

Anti-affinity strategies guarantee cluster availability in different ways.



NOTE

The equations in the following section apply only if the number of OpenShift nodes or zones is greater than the number of Data Grid nodes.

Scheduling pods on different OpenShift nodes

Provides tolerance of **x** node failures for the following types of cache:

- Replicated: **x = spec.replicas - 1**
- Distributed: **x = num_owners - 1**

Scheduling pods across multiple zones

Provides tolerance of **x** zone failures when **x** zones exist for the following types of cache:

- Replicated: **x = spec.replicas - 1**
- Distributed: **x = num_owners - 1**



NOTE

spec.replicas

Defines the number of pods in each Data Grid cluster.

num_owners

Is the cache configuration attribute that defines the number of replicas for each entry in the cache.

16.2. CONFIGURING ANTI-AFFINITY

Specify where OpenShift schedules pods for your Data Grid clusters to ensure availability.

Procedure

1. Add the **spec.scheduling.affinity** block to your **Infinispan** CR.
2. Configure anti-affinity strategies as necessary.
3. Apply your **Infinispan** CR.

16.2.1. Anti-affinity strategy configurations

Configure anti-affinity strategies in your **Infinispan** CR to control where OpenShift schedules Data Grid replica pods.

Topology keys	Description
topologyKey: "topology.kubernetes.io/zone"	Schedules Data Grid replica pods across multiple zones.
topologyKey: "kubernetes.io/hostname"	Schedules Data Grid replica pods on different OpenShift nodes.

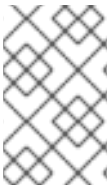
Schedule pods on different OpenShift nodes
The following is the anti-affinity strategy that Data Grid Operator uses if you do not configure the **spec.scheduling.affinity** field in your **Infinispan** CR:

```
spec:
  scheduling:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: infinispan-pod
                clusterName: <cluster_name>
                infinispan_cr: <cluster_name>
            topologyKey: "kubernetes.io/hostname"
```

Requiring different nodes
In the following example, OpenShift does not schedule Data Grid pods if different nodes are not available:

```
spec:
  scheduling:
    affinity:
      podAntiAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
```

```
    - labelSelector:
        matchLabels:
          app: infinispan-pod
          clusterName: <cluster_name>
          infinispan_cr: <cluster_name>
      topologyKey: "topology.kubernetes.io/hostname"
```



NOTE

To ensure that you can schedule Data Grid replica pods on different OpenShift nodes, the number of OpenShift nodes available must be greater than the value of **spec.replicas**.

Schedule pods across multiple OpenShift zones

The following example prefers multiple zones when scheduling pods but schedules Data Grid replica pods on different OpenShift nodes if it is not possible to schedule across zones:

```
spec:
  scheduling:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: infinispan-pod
                clusterName: <cluster_name>
                infinispan_cr: <cluster_name>
            topologyKey: "topology.kubernetes.io/zone"
        - weight: 90
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: infinispan-pod
                clusterName: <cluster_name>
                infinispan_cr: <cluster_name>
            topologyKey: "kubernetes.io/hostname"
```

Requiring multiple zones

The following example uses the zone strategy only when scheduling Data Grid replica pods:

```
spec:
  scheduling:
    affinity:
      podAntiAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: infinispan-pod
              clusterName: <cluster_name>
              infinispan_cr: <cluster_name>
          topologyKey: "topology.kubernetes.io/zone"
```

CHAPTER 17. AUTO SCALING

Kubernetes includes the **HorizontalPodAutoscaler** which allows StatefulSets or Deployments to be automatically scaled up or down based upon specified metrics. The Infinispan CR exposes the **.status.scale** sub-resource, which enables **HorizontalPodAutoscaler** resources to target the Infinispan CR.

Before defining a **HorizontalPodAutoscaler** configuration, consider the types of Data Grid caches that you define. Distributed and Replicated caches have very different scaling requirements, so defining a **HorizontalPodAutoscaler** for server's running a combination of these cache types may not be advantageous. For example, defining a **HorizontalPodAutoscaler** that scales when memory usage reaches a certain percentage will allow overall cache capacity to be increased when defining Distributed caches as cache entries are spread across pods, however it will not work with replicated cache as every pod hosts all cache entries. Conversely, configuring a **HorizontalPodAutoscaler** based upon CPU usage will be more beneficial for clusters with replicated cache as every pod contains all cache entries and so distributing read requests across additional nodes will allow a greater number of requests to be processed simultaneously.

17.1. CONFIGURING HORIZONTALPODAUTOSCALER

Create a HorizontalPodAutoScaler resource that targets your Infinispan CR.

Procedure

1. Define a **HorizontalPodAutoscaler** resource in the same namespace as your **Infinispan** CR

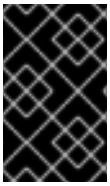
```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: infinispan-auto
spec:
  scaleTargetRef:
    apiVersion: infinispan.org/v1
    kind: Infinispan
    name: example 1
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

1 The name of your **Infinispan** CR



NOTE

If using metric resource of type **cpu** or **memory**, you must configure request/limits for this resource in your **Infinispan** CR.



IMPORTANT

HorizontalPodAutoscaler should be removed when upgrading a Data Grid cluster, as the automatic scaling will cause the upgrade process to enter unexpected state, as the Operator needs to scale the cluster down to 0 pods.

CHAPTER 18. CREATING CACHES WITH DATA GRID OPERATOR

Use **Cache** CRs to add cache configuration with Data Grid Operator and control how Data Grid stores your data.

18.1. DATA GRID CACHES

Cache configuration defines the characteristics and features of the data store and must be valid with the Data Grid schema. Data Grid recommends creating standalone files in XML or JSON format that define your cache configuration. You should separate Data Grid configuration from application code for easier validation and to avoid the situation where you need to maintain XML snippets in Java or some other client language.

To create caches with Data Grid clusters running on OpenShift, you should:

- Use **Cache** CR as the mechanism for creating caches through the OpenShift front end.
- Use **Batch** CR to create multiple caches at a time from standalone configuration files.
- Access Data Grid Console and create caches in XML or JSON format.

You can use Hot Rod or HTTP clients but Data Grid recommends **Cache** CR or **Batch** CR unless your specific use case requires programmatic remote cache creation.

Cache CRs

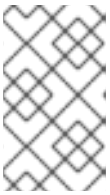
- **Cache** CRs apply to Data Grid service pods only.
- Each **Cache** CR corresponds to a single cache on the Data Grid cluster.

18.2. CREATING CACHES WITH THE CACHE CR

Complete the following steps to create caches on Data Grid service clusters using valid configuration in XML or YAML format.

Procedure

1. Create a **Cache** CR with a unique value in the **metadata.name** field.
2. Specify the target Data Grid cluster with the **spec.clusterName** field.
3. Name your cache with the **spec.name** field.



NOTE

The **name** attribute in the cache configuration does not take effect. If you do not specify a name with the **spec.name** field then the cache uses the value of the **metadata.name** field.

4. Add a cache configuration with the **spec.template** field.
5. Apply the **Cache** CR, for example:

```
oc apply -f mycache.yaml
cache.infinispan.org/mycachedefinition created
```

Cache CR examples

XML

```
apiVersion: infinispan.org/v2alpha1
kind: Cache
metadata:
  name: mycachedefinition
spec:
  clusterName: infinispan
  name: myXMLcache
  template: <distributed-cache mode="SYNC" statistics="true"><encoding media-type="application/x-
protostream"/><persistence><file-store/></persistence></distributed-cache>
```

YAML

```
apiVersion: infinispan.org/v2alpha1
kind: Cache
metadata:
  name: mycachedefinition
spec:
  clusterName: infinispan
  name: myYAMLcache
  template: |-
    distributedCache:
      mode: "SYNC"
      owners: "2"
      statistics: "true"
      encoding:
        mediaType: "application/x-protostream"
      persistence:
        fileStore: ~
```

18.3. UPDATING CACHES WITH THE CACHE CR

You can control how Data Grid Operator handles modifications to the cache configuration in the **Cache** CR.

Data Grid Operator attempts to update the cache configuration on the Data Grid Server at runtime. If the update fails, Data Grid Operator uses one of the following strategies:

retain strategy

The Operator updates the status of the **Cache** CR to **Ready=False**. You can manually delete the **Cache** CR and create a new cache configuration. This is the default strategy.

recreate strategy

The Operator deletes the cache from the Data Grid cluster and creates a new cache with the latest **spec.template** value from the **Cache** CR.



IMPORTANT

Configure the **recreate** strategy only if your deployment can tolerate data loss.

Prerequisites

- Have a valid **Cache** CR.

Procedure

1. Use the **spec.updates.strategy** field to set the **Cache** CR strategy.

   mycache.yaml

   ```
   spec:
     updates:
       strategy: recreate
   ```

2. Apply changes to the **Cache** CR, for example:

   ```
   oc apply -f mycache.yaml
   ```

18.4. ADDING PERSISTENT CACHE STORES

You can add persistent cache stores to Data Grid service pods to save data to the persistent volume.

Data Grid creates a Single File cache store, **.dat** file, in the **/opt/infinispan/server/data** directory.

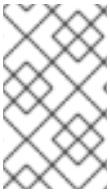
Procedure

- Add the **<file-store/>** element to the **persistence** configuration in your Data Grid cache, as in the following example:

  ```
  <distributed-cache name="persistent-cache" mode="SYNC">
    <encoding media-type="application/x-protostream"/>
    <persistence>
      <file-store/>
    </persistence>
  </distributed-cache>
  ```

CHAPTER 19. RUNNING BATCH OPERATIONS

Data Grid Operator provides a **Batch** CR that lets you create Data Grid resources in bulk. **Batch** CR uses the Data Grid command line interface (CLI) in batch mode to carry out sequences of operations.



NOTE

Modifying a **Batch** CR instance has no effect. Batch operations are "one-time" events that modify Data Grid resources. To update **.spec** fields for the CR, or when a batch operation fails, you must create a new instance of the **Batch** CR.

19.1. RUNNING INLINE BATCH OPERATIONS

Include your batch operations directly in a **Batch** CR if they do not require separate configuration artifacts.

Procedure

1. Create a **Batch** CR.
   a. Specify the name of the Data Grid cluster where you want the batch operations to run as the value of the **spec.cluster** field.
   b. Add each CLI command to run on a line in the **spec.config** field.

```
apiVersion: infinispan.org/v2alpha1
kind: Batch
metadata:
  name: mybatch
spec:
  cluster: infinispan
  config: |
    create counter --concurrency-level=1 --initial-value=5 --storage=VOLATILE --
type=weak batch-counter
```

2. Apply your **Batch** CR.

```
oc apply -f mybatch.yaml
```

3. Wait for the **Batch** CR to succeed.

```
oc wait --for=jsonpath='{.status.phase}'=Succeeded Batch/mybatch
```

19.2. CREATING CONFIGMAPS FOR BATCH OPERATIONS

Create a **ConfigMap** so that additional files, such as Data Grid cache configuration, are available for batch operations.

Prerequisites

For demonstration purposes, you should add some configuration artifacts to your host filesystem before you start the procedure:

- Create a **/tmp/mybatch** directory where you can add some files.

```
mkdir -p /tmp/mybatch
```

- Create a Data Grid cache configuration.

```
cat > /tmp/mybatch/mycache.xml<<EOF
<distributed-cache name="mycache" mode="SYNC">
  <encoding media-type="application/x-protostream"/>
  <memory max-count="1000000" when-full="REMOVE"/>
</distributed-cache>
EOF
```

Procedure

1. Create a **batch** file that contains all commands you want to run.
   For example, the following **batch** file creates a cache named "mycache" and adds two entries to it:

```
create cache mycache --file=/etc/batch/mycache.xml
put --cache=mycache hello world
put --cache=mycache hola mundo
```



IMPORTANT

The **ConfigMap** is mounted in Data Grid pods at **/etc/batch**. You must prepend all **--file=** directives in your batch operations with that path.

2. Ensure all configuration artifacts that your batch operations require are in the same directory as the **batch** file.

```
ls /tmp/mybatch

batch
mycache.xml
```

3. Create a **ConfigMap** from the directory.

```
oc create configmap mybatch-config-map --from-file=/tmp/mybatch
```

19.3. RUNNING BATCH OPERATIONS WITH CONFIGMAPS

Run batch operations that include configuration artifacts.

Prerequisites

- Create a **ConfigMap** that contains any files your batch operations require.

Procedure

1. Create a **Batch** CR that specifies the name of a Data Grid cluster as the value of the **spec.cluster** field.
2. Set the name of the **ConfigMap** that contains your **batch** file and configuration artifacts with the **spec.configMap** field.

```
cat > mybatch.yaml<<EOF
apiVersion: infinispan.org/v2alpha1
kind: Batch
metadata:
  name: mybatch
spec:
  cluster: infinispan
  configMap: mybatch-config-map
EOF
```

3. Apply your **Batch** CR.

```
oc apply -f mybatch.yaml
```

4. Wait for the **Batch** CR to succeed.

```
oc wait --for=jsonpath='{.status.phase}'=Succeeded Batch/mybatch
```

19.4. BATCH STATUS MESSAGES

Verify and troubleshoot batch operations with the **status.Phase** field in the **Batch** CR.

Phase	Description
Succeeded	All batch operations have completed successfully.
Initializing	Batch operations are queued and resources are initializing.
Initialized	Batch operations are ready to start.
Running	Batch operations are in progress.
Failed	One or more batch operations were not successful.

Failed operations

Batch operations are not atomic. If a command in a batch script fails, it does not affect the other operations or cause them to rollback.

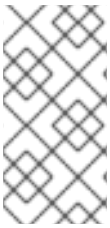


NOTE

If your batch operations have any server or syntax errors, you can view log messages in the **Batch** CR in the **status.Reason** field.

19.5. EXAMPLE BATCH OPERATIONS

Use these example batch operations as starting points for creating and modifying Data Grid resources with the **Batch** CR.



NOTE

You can pass configuration files to Data Grid Operator only via a **ConfigMap**.

The **ConfigMap** is mounted in Data Grid pods at **/etc/batch** so you must prepend all **--file=** directives with that path.

19.5.1. Caches

- Create multiple caches from configuration files.

```
echo "creating caches..."
create cache sessions --file=/etc/batch/infinispan-prod-sessions.xml
create cache tokens --file=/etc/batch/infinispan-prod-tokens.xml
create cache people --file=/etc/batch/infinispan-prod-people.xml
create cache books --file=/etc/batch/infinispan-prod-books.xml
create cache authors --file=/etc/batch/infinispan-prod-authors.xml
echo "list caches in the cluster"
ls caches
```

- Create a template from a file and then create caches from the template.

```
echo "creating caches..."
create cache mytemplate --file=/etc/batch/mycache.xml
create cache sessions --template=mytemplate
create cache tokens --template=mytemplate
echo "list caches in the cluster"
ls caches
```

19.5.2. Counters

Use the **Batch** CR to create multiple counters that can increment and decrement to record the count of objects.

You can use counters to generate identifiers, act as rate limiters, or track the number of times a resource is accessed.

```
echo "creating counters..."
create counter --concurrency-level=1 --initial-value=5 --storage=PERSISTENT --type=weak
mycounter1
create counter --initial-value=3 --storage=PERSISTENT --type=strong mycounter2
create counter --initial-value=13 --storage=PERSISTENT --type=strong --upper-bound=10
mycounter3
echo "list counters in the cluster"
ls counters
```

19.5.3. Protobuf schema

Register Protobuf schema to query values in caches. Protobuf schema (**.proto** files) provide metadata about custom entities and controls field indexing.

```
echo "creating schema..."
schema --upload=person.proto person.proto
schema --upload=book.proto book.proto
schema --upload=author.proto book.proto
echo "list Protobuf schema"
ls schemas
```

19.5.4. Tasks

Upload tasks that implement **org.infinispan.tasks.ServerTask** or scripts that are compatible with the **javax.script** scripting API.

```
echo "creating tasks..."
task upload --file=/etc/batch/myfirstscript.js myfirstscript
task upload --file=/etc/batch/mysecondscript.js mysecondscript
task upload --file=/etc/batch/mythirdscript.js mythirdscript
echo "list tasks"
ls tasks
```

Additional resources

- Data Grid CLI Guide

CHAPTER 20. BACKING UP AND RESTORING DATA GRID CLUSTERS

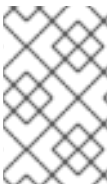
Data Grid Operator lets you back up and restore Data Grid cluster state for disaster recovery and to migrate Data Grid resources between clusters.

20.1. BACKUP AND RESTORE CRS

Backup and **Restore** CRs save in-memory data at runtime so you can easily recreate Data Grid clusters.

Applying a **Backup** or **Restore** CR creates a new pod that joins the Data Grid cluster as a zero-capacity member, which means it does not require cluster rebalancing or state transfer to join.

For backup operations, the pod iterates over cache entries and other resources and creates an archive, a **.zip** file, in the **/opt/infinispan/backups** directory on the persistent volume (PV).



NOTE

Performing backups does not significantly impact performance because the other pods in the Data Grid cluster only need to respond to the backup pod as it iterates over cache entries.

For restore operations, the pod retrieves Data Grid resources from the archive on the PV and applies them to the Data Grid cluster.

When either the backup or restore operation completes, the pod leaves the cluster and is terminated.

Reconciliation

Data Grid Operator does not reconcile **Backup** and **Restore** CRs which mean that backup and restore operations are "one-time" events.

Modifying an existing **Backup** or **Restore** CR instance does not perform an operation or have any effect. If you want to update **.spec** fields, you must create a new instance of the **Backup** or **Restore** CR.

20.2. BACKING UP DATA GRID CLUSTERS

Create a backup file that stores Data Grid cluster state to a persistent volume.

Prerequisites

- Create an **Infinispan** CR with **spec.service.type: DataGrid**.
- Ensure there are no active client connections to the Data Grid cluster.
  Data Grid backups do not provide snapshot isolation and data modifications are not written to the archive after the cache is backed up.
  To archive the exact state of the cluster, you should always disconnect any clients before you back it up.

Procedure

1. Name the **Backup** CR with the **metadata.name** field.

2. Specify the Data Grid cluster to backup with the **spec.cluster** field.

3. Configure the persistent volume claim (PVC) that adds the backup archive to the persistent volume (PV) with the **spec.volume.storage** and **spec.volume.storage.storageClassName** fields.

```
apiVersion: infinispan.org/v2alpha1
kind: Backup
metadata:
  name: my-backup
spec:
  cluster: source-cluster
  volume:
    storage: 1Gi
    storageClassName: my-storage-class
```

4. Optionally include **spec.resources** fields to specify which Data Grid resources you want to back up.
   If you do not include any **spec.resources** fields, the **Backup** CR creates an archive that contains all Data Grid resources. If you do specify **spec.resources** fields, the **Backup** CR creates an archive that contains those resources only.

```
spec:
  ...
  resources:
    templates:
      - distributed-sync-prod
      - distributed-sync-dev
    caches:
      - cache-one
      - cache-two
    counters:
      - counter-name
    protoSchemas:
      - authors.proto
      - books.proto
    tasks:
      - wordStream.js
```

   You can also use the * wildcard character as in the following example:

```
spec:
  ...
  resources:
    caches:
      - "*"
    protoSchemas:
      - "*"
```

5. Apply your **Backup** CR.

```
oc apply -f my-backup.yaml
```

Verification

1. Check that the **status.phase** field has a status of **Succeeded** in the **Backup** CR and that Data Grid logs have the following message:

```
ISPN005044: Backup file created 'my-backup.zip'
```

2. Run the following command to check that the backup is successfully created:

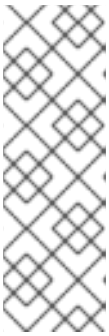
```
oc describe Backup my-backup
```

20.3. RESTORING DATA GRID CLUSTERS

Restore Data Grid cluster state from a backup archive.

Prerequisites

- Create a **Backup** CR on a source cluster.
- Create a target Data Grid cluster of Data Grid service pods.



NOTE

If you restore an existing cache, the operation overwrites the data in the cache but not the cache configuration.

For example, you back up a distributed cache named **mycache** on the source cluster. You then restore **mycache** on a target cluster where it already exists as a replicated cache. In this case, the data from the source cluster is restored and **mycache** continues to have a replicated configuration on the target cluster.

- Ensure there are no active client connections to the target Data Grid cluster you want to restore. Cache entries that you restore from a backup can overwrite more recent cache entries. For example, a client performs a **cache.put(k=2)** operation and you then restore a backup that contains **k=1**.

Procedure

1. Name the **Restore** CR with the **metadata.name** field.
2. Specify a **Backup** CR to use with the **spec.backup** field.
3. Specify the Data Grid cluster to restore with the **spec.cluster** field.

```
apiVersion: infinispan.org/v2alpha1
kind: Restore
metadata:
  name: my-restore
spec:
  backup: my-backup
  cluster: target-cluster
```

4. Optionally add the **spec.resources** field to restore specific resources only.

```
spec:
  ...
  resources:
    templates:
      - distributed-sync-prod
      - distributed-sync-dev
    caches:
      - cache-one
      - cache-two
    counters:
      - counter-name
    protoSchemas:
      - authors.proto
      - books.proto
    tasks:
      - wordStream.js
```

5. Apply your **Restore** CR.

```
oc apply -f my-restore.yaml
```

Verification

- Check that the **status.phase** field has a status of **Succeeded** in the **Restore** CR and that Data Grid logs have the following message:

```
ISPN005045: Restore 'my-backup' complete
```

You should then open the Data Grid Console or establish a CLI connection to verify data and Data Grid resources are restored as expected.

20.4. BACKUP AND RESTORE STATUS

Backup and **Restore** CRs include a **status.phase** field that provides the status for each phase of the operation.

Status	Description
Initializing	The system has accepted the request and the controller is preparing the underlying resources to create the pod.
Initialized	The controller has prepared all underlying resources successfully.
Running	The pod is created and the operation is in progress on the Data Grid cluster.
Succeeded	The operation has completed successfully on the Data Grid cluster and the pod is terminated.

Status	Description
Failed	The operation did not successfully complete and the pod is terminated.
Unknown	The controller cannot obtain the status of the pod or determine the state of the operation. This condition typically indicates a temporary communication error with the pod.

20.4.1. Handling failed backup and restore operations

If the **status.phase** field of the **Backup** or **Restore** CR is **Failed**, you should examine pod logs to determine the root cause before you attempt the operation again.

Procedure

1. Examine the logs for the pod that performed the failed operation.
   Pods are terminated but remain available until you delete the **Backup** or **Restore** CR.

   ```
   oc logs <backup|restore_pod_name>
   ```

2. Resolve any error conditions or other causes of failure as indicated by the pod logs.
3. Create a new instance of the **Backup** or **Restore** CR and attempt the operation again.

CHAPTER 21. DEPLOYING CUSTOM CODE TO DATA GRID

Add custom code, such as scripts and event listeners, to your Data Grid clusters.

Before you can deploy custom code to Data Grid clusters, you need to make it available. To do this you can copy artifacts from a persistent volume (PV), download artifacts from an HTTP or FTP server, or use both methods.

21.1. COPYING CODE ARTIFACTS TO DATA GRID CLUSTERS

Adding your artifacts to a persistent volume (PV) and then copy them to Data Grid pods.

This procedure explains how to use a temporary pod that mounts a persistent volume claim (PVC) that:

- Lets you add code artifacts to the PV (perform a write operation).
- Allows Data Grid pods to load code artifacts from the PV (perform a read operation).

To perform these read and write operations, you need certain PV access modes. However, support for different PVC access modes is platform dependent.

It is beyond the scope of this document to provide instructions for creating PVCs with different platforms. For simplicity, the following procedure shows a PVC with the **ReadWriteMany** access mode.

In some cases only the **ReadOnlyMany** or **ReadWriteOnce** access modes are available. You can use a combination of those access modes by reclaiming and reusing PVCs with the same **spec.volumeName**.



NOTE

Using **ReadWriteOnce** access mode results in all Data Grid pods in a cluster being scheduled on the same OpenShift node.

Procedure

1. Change to the namespace for your Data Grid cluster.

```
oc project rhdg-namespace
```

2. Create a PVC for your custom code artifacts, for example:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: datagrid-libs
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 100Mi
```

3. Apply your PVC.

```
oc apply -f datagrid-libs.yaml
```

4. Create a pod that mounts the PVC, for example:

```
apiVersion: v1
kind: Pod
metadata:
  name: datagrid-libs-pod
spec:
  securityContext:
    fsGroup: 2000
  volumes:
    - name: lib-pv-storage
      persistentVolumeClaim:
        claimName: datagrid-libs
  containers:
    - name: lib-pv-container
      image: registry.redhat.io/datagrid/datagrid-8-rhel9:8.6
      volumeMounts:
        - mountPath: /tmp/libs
          name: lib-pv-storage
```

5. Add the pod to the Data Grid namespace and wait for it to be ready.

```
oc apply -f datagrid-libs-pod.yaml
oc wait --for=condition=ready --timeout=2m pod/datagrid-libs-pod
```

6. Copy your code artifacts to the pod so that they are loaded into the PVC.
   For example to copy code artifacts from a local **libs** directory, do the following:

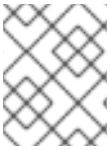
```
oc cp --no-preserve=true libs datagrid-libs-pod:/tmp/
```

7. Delete the pod.

```
oc delete pod datagrid-libs-pod
```

   Specify the persistent volume with **spec.dependencies.volumeClaimName** in your **Infinispan** CR and then apply the changes.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  dependencies:
    volumeClaimName: datagrid-libs
  service:
    type: DataGrid
```



NOTE

If you update your custom code on the persistent volume, you must restart the Data Grid cluster so it can load the changes.

Additional resources

- Configuring persistent storage
- Persistent Volumes
- Access Modes
- How to manually reclaim and reuse OpenShift Persistent volumes that are "Released" (Red Hat Knowledgebase)

21.2. DOWNLOADING CODE ARTIFACTS

Add your artifacts to an HTTP or FTP server so that Data Grid Operator downloads them to the **{lib_path}** directory on each Data Grid node.

When downloading files, Data Grid Operator can automatically detect the file type. Data Grid Operator also extracts archived files, such as **zip** or **tgz**, to the filesystem after the download completes.

You can also download Maven artifacts using the **groupId:artifactId:version** format, for example **org.postgresql:postgresql:42.3.1**.



NOTE

Each time Data Grid Operator creates a Data Grid node it downloads the artifacts to the node.

Prerequisites

- Host your code artifacts on an HTTP or FTP server or publish them to a maven repository.

Procedure

1. Add the **spec.dependencies.artifacts** field to your **Infinispan** CR.
2. Do one of the following:
   - Specify the location of the file to download via **HTTP** or **FTP** as the value of the **spec.dependencies.artifacts.url** field.
   - Provide the Maven artifact to download with the **groupId:artifactId:version** format as the value of the **spec.dependencies.artifacts.maven** field.
3. Optionally specify a checksum to verify the integrity of the download with the **spec.dependencies.artifacts.hash** field.
   The **hash** field requires a value is in the format of **<algorithm>:<checksum>** where **<algorithm>** is **sha1|sha224|sha256|sha384|sha512|md5**.

```
apiVersion: infinispan.org/v1
kind: Infinispan
metadata:
  name: infinispan
spec:
  replicas: 2
  dependencies:
```

```
    artifacts:
      - url: http://example.com:8080/path
        hash:
sha256:596408848b56b5a23096baa110cd8b633c9a9aef2edd6b38943ade5b4edcd686
  service:
    type: DataGrid
```

4. Apply the changes.

CHAPTER 22. ESTABLISHING REMOTE CLIENT CONNECTIONS

Connect to Data Grid clusters from the Data Grid Console, Command Line Interface (CLI), and remote clients.

22.1. CLIENT CONNECTION DETAILS

Client connections to Data Grid require the following information:

- Hostname
- Port
- Authentication credentials, if required
- TLS certificate, if you use encryption

Hostnames

The hostname you use depends on whether clients are running on the same OpenShift cluster as Data Grid.

Client applications running on the same OpenShift cluster use the internal service name for the Data Grid cluster.

```
metadata:
  name: infinispan
```

Client applications running on a different OpenShift, or outside OpenShift, use a hostname that depends on how Data Grid is exposed on the network.

A **LoadBalancer** service uses the URL for the load balancer. A **NodePort** service uses the node hostname. An Red Hat OpenShift **Route** uses either a custom hostname that you define or a hostname that the system generates.

Ports

Client connections on OpenShift and a through **LoadBalancer** service use port **11222**.

NodePort services use a port in the range of **30000** to **60000**. Routes use either port **80** (unencrypted) or **443** (encrypted).

Additional resources

- Configuring Network Access to Data Grid
- Retrieving Credentials
- Retrieving TLS Certificates

22.2. CONNECTING TO DATA GRID CLUSTERS WITH REMOTE SHELLS

Start a remote shell session to Data Grid clusters and use the command line interface (CLI) to work with Data Grid resources and perform administrative operations.

Prerequisites

- Have **kubectl-infinispan** on your **PATH**.
- Have valid Data Grid credentials.

Procedure

1. Run the **infinispan shell** command to connect to your Data Grid cluster.

```
oc infinispan shell <cluster_name>
```



NOTE

If you have access to authentication secrets and there is only one Data Grid user the **kubectl-infinispan** plugin automatically detects your credentials and authenticates to Data Grid. If your deployment has multiple Data Grid credentials, specify a user with the **--username** argument and enter the corresponding password when prompted.

2. Perform CLI operations as required.

TIP

Press the tab key or use the **--help** argument to view available options and help text.

3. Use the **quit** command to end the remote shell session.

Additional resources

- Using the Data Grid Command Line Interface

22.3. ACCESSING DATA GRID CONSOLE

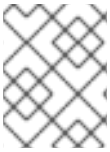
Access the console to create caches, perform adminstrative operations, and monitor your Data Grid clusters.

Prerequisites

- Expose Data Grid on the network so you can access the console through a browser.
  For example, configure a **LoadBalancer** service or create a **Route**.

Procedure

- Access the console from any browser at **$HOSTNAME:$PORT**.
  Replace **$HOSTNAME:$PORT** with the network location where Data Grid is available.



NOTE

The Data Grid Console should only be accessed via OpenShift services or an OpenShift **Route** exposing port 11222.

22.4. HOT ROD CLIENTS

Hot Rod is a binary TCP protocol that Data Grid provides for high-performance data transfer capabilities with remote clients.

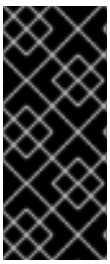
Client intelligence

The Hot Rod protocol includes a mechanism that provides clients with an up-to-date view of the cache topology. Client intelligence improves performance by reducing the number of network hops for read and write operations.

Clients running in the same OpenShift cluster can access internal IP addresses for Data Grid pods so you can use any client intelligence.

HASH_DISTRIBUTION_AWARE is the default intelligence mechanism and enables clients to route requests to primary owners, which provides the best performance for Hot Rod clients.

Clients running on a different OpenShift, or outside OpenShift, can access Data Grid by using a **LoadBalancer**, **NodePort**, or OpenShift **Route**.



IMPORTANT

Hot Rod client connections via OpenShift **Route** require encryption. You must configure TLS with SNI otherwise the Hot Rod connection fails.

For unencrypted Hot Rod client connections, you must use a **LoadBalancer** service or a **NodePort** service.

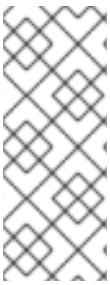
Hot Rod clients must use **BASIC** intelligence in the following situations:

- Connecting to Data Grid through a **LoadBalancer** service, a **NodePort** service, or an OpenShift **Route**.
- Failing over to a different OpenShift cluster when using cross-site replication.

OpenShift cluster administrators can define network policies that restrict traffic to Data Grid. In some cases network isolation policies can require you to use **BASIC** intelligence even when clients are running in the same OpenShift cluster but a different namespace.

22.4.1. Hot Rod client configuration API

You can programmatically configure Hot Rod client connections with the **ConfigurationBuilder** interface.



NOTE

Replace **$SERVICE_HOSTNAME** in the following examples with the internal service name of your Data Grid cluster.

```
metadata:
  name: infinispan
```

On OpenShift

ConfigurationBuilder

```
import org.infinispan.client.hotrod.configuration.ConfigurationBuilder;
import org.infinispan.client.hotrod.configuration.SaslQop;
import org.infinispan.client.hotrod.impl.ConfigurationProperties;
...

ConfigurationBuilder builder = new ConfigurationBuilder();
    builder.addServer()
            .host("$HOSTNAME")
            .port(ConfigurationProperties.DEFAULT_HOTROD_PORT)
           .security().authentication()
            .username("username")
            .password("changeme")
            .realm("default")
            .saslQop(SaslQop.AUTH)
            .saslMechanism("SCRAM-SHA-512")
           .ssl()
            .sniHostName("$SERVICE_HOSTNAME")
            .trustStoreFileName("/var/run/secrets/kubernetes.io/serviceaccount/service-ca.crt")
            .trustStoreType("pem");
```

hotrod-client.properties

```
# Connection
infinispan.client.hotrod.server_list=$HOSTNAME:$PORT

# Authentication
infinispan.client.hotrod.use_auth=true
infinispan.client.hotrod.auth_username=developer
infinispan.client.hotrod.auth_password=$PASSWORD
infinispan.client.hotrod.auth_server_name=$CLUSTER_NAME
infinispan.client.hotrod.sasl_properties.javax.security.sasl.qop=auth
infinispan.client.hotrod.sasl_mechanism=SCRAM-SHA-512

# Encryption
infinispan.client.hotrod.sni_host_name=$SERVICE_HOSTNAME
infinispan.client.hotrod.trust_store_file_name=/var/run/secrets/kubernetes.io/serviceaccount/service-
ca.crt
infinispan.client.hotrod.trust_store_type=pem
```

Outside OpenShift

ConfigurationBuilder

```
import org.infinispan.client.hotrod.configuration.ClientIntelligence;
import org.infinispan.client.hotrod.configuration.ConfigurationBuilder;
import org.infinispan.client.hotrod.configuration.SaslQop;
...

ConfigurationBuilder builder = new ConfigurationBuilder();
    builder.addServer()
            .host("$HOSTNAME")
            .port("$PORT")
           .security().authentication()
```

```
            .username("username")
            .password("changeme")
            .realm("default")
            .saslQop(SaslQop.AUTH)
            .saslMechanism("SCRAM-SHA-512")
          .ssl()
            .sniHostName("$SERVICE_HOSTNAME")
            //Create a client trust store with tls.crt from your project.
            .trustStoreFileName("/path/to/truststore.pkcs12")
            .trustStorePassword("trust_store_password")
            .trustStoreType("PCKS12");
    builder.clientIntelligence(ClientIntelligence.BASIC);
```

hotrod-client.properties

```
# Connection
infinispan.client.hotrod.server_list=$HOSTNAME:$PORT

# Client intelligence
infinispan.client.hotrod.client_intelligence=BASIC

# Authentication
infinispan.client.hotrod.use_auth=true
infinispan.client.hotrod.auth_username=developer
infinispan.client.hotrod.auth_password=$PASSWORD
infinispan.client.hotrod.auth_server_name=$CLUSTER_NAME
infinispan.client.hotrod.sasl_properties.javax.security.sasl.qop=auth
infinispan.client.hotrod.sasl_mechanism=SCRAM-SHA-512

# Encryption
infinispan.client.hotrod.sni_host_name=$SERVICE_HOSTNAME
# Create a client trust store with tls.crt from your project.
infinispan.client.hotrod.trust_store_file_name=/path/to/truststore.pkcs12
infinispan.client.hotrod.trust_store_password=trust_store_password
infinispan.client.hotrod.trust_store_type=PCKS12
```

22.4.2. Configuring Hot Rod clients for certificate authentication

If you enable client certificate authentication, clients must present valid certificates when negotiating connections with Data Grid.

Validate strategy

If you use the **Validate** strategy, you must configure clients with a keystore so they can present signed certificates. You must also configure clients with Data Grid credentials and any suitable authentication mechanism.

Authenticate strategy

If you use the **Authenticate** strategy, you must configure clients with a keystore that contains signed certificates and valid Data Grid credentials as part of the distinguished name (DN). Hot Rod clients must also use the **EXTERNAL** authentication mechanism.



NOTE

If you enable security authorization, you should assign the Common Name (CN) from the client certificate a role with the appropriate permissions.

The following example shows a Hot Rod client configuration for client certificate authentication with the **Authenticate** strategy:

```
import org.infinispan.client.hotrod.configuration.ConfigurationBuilder;
...

ConfigurationBuilder builder = new ConfigurationBuilder();
    builder.security()
          .authentication()
           .saslMechanism("EXTERNAL")
          .ssl()
           .keyStoreFileName("/path/to/keystore")
           .keyStorePassword("keystorepassword".toCharArray())
           .keyStoreType("PCKS12");
```

22.4.3. Creating caches from Hot Rod clients

You can remotely create caches on Data Grid clusters running on OpenShift with Hot Rod clients. However, Data Grid recommends that you create caches using Data Grid Console, the CLI, or with **Cache** CRs instead of with Hot Rod clients.

Programmatically creating caches

The following example shows how to add cache configurations to the **ConfigurationBuilder** and then create them with the **RemoteCacheManager**:

```
import org.infinispan.client.hotrod.DefaultTemplate;
import org.infinispan.client.hotrod.RemoteCache;
import org.infinispan.client.hotrod.RemoteCacheManager;
...

    builder.remoteCache("my-cache")
          .templateName(DefaultTemplate.DIST_SYNC);
    builder.remoteCache("another-cache")
          .configuration("<infinispan><cache-container><distributed-cache name=\"another-cache\">
<encoding media-type=\"application/x-protostream\"/></distributed-cache></cache-container>
</infinispan>");
    try (RemoteCacheManager cacheManager = new RemoteCacheManager(builder.build())) {
    // Get a remote cache that does not exist.
    // Rather than return null, create the cache from a template.
    RemoteCache<String, String> cache = cacheManager.getCache("my-cache");
    // Store a value.
    cache.put("hello", "world");
    // Retrieve the value and print it.
    System.out.printf("key = %s\n", cache.get("hello"));
```

This example shows how to create a cache named CacheWithXMLConfiguration using the **XMLStringConfiguration()** method to pass the cache configuration as XML:

```
import org.infinispan.client.hotrod.RemoteCacheManager;
```

```
import org.infinispan.commons.configuration.XMLStringConfiguration;
...

private void createCacheWithXMLConfiguration() {
   String cacheName = "CacheWithXMLConfiguration";
   String xml = String.format("<distributed-cache name=\"%s\">" +
                              "<encoding media-type=\"application/x-protostream\"/>" +
                              "<locking isolation=\"READ_COMMITTED\"/>" +
                              "<transaction mode=\"NON_XA\"/>" +
                              "<expiration lifespan=\"60000\" interval=\"20000\"/>" +
                             "</distributed-cache>"
                             , cacheName);
   manager.administration().getOrCreateCache(cacheName, new XMLStringConfiguration(xml));
   System.out.println("Cache with configuration exists or is created.");
}
```

Using Hot Rod client properties

When you invoke **cacheManager.getCache()** calls for named caches that do not exist, Data Grid creates them from the Hot Rod client properties instead of returning null.

Add cache configuration to **hotrod-client.properties** as in the following example:

```
# Add cache configuration
infinispan.client.hotrod.cache.my-cache.configuration=<infinispan><cache-container><distributed-cache name=\"my-cache\"/></cache-container></infinispan>
infinispan.client.hotrod.cache.another-cache.configuration_uri=file:/path/to/configuration.xml
```

22.5. ACCESSING THE REST API

Data Grid provides a RESTful interface that you can interact with using HTTP clients.

Prerequisites

- Expose Data Grid on the network so you can access the REST API.
  For example, configure a **LoadBalancer** service or create a **Route**.

Procedure

- Access the REST API with any HTTP client at **$HOSTNAME:$PORT/rest/v2**.
  Replace **$HOSTNAME:$PORT** with the network location where Data Grid listens for client connections.

Additional resources

- Data Grid REST API